Framework for Evaluating Explainability Methods in Artificial Intelligence

User Manual
XAIMetrics version 1.0
July 24, 2026

Maria Teresa Alba Rueda
Amelia Zafra Gómez

Contents

Acronyms

XAI: Explainable Artificial Intelligence

AI: Artificial Intelligence

CHAPTER

1

Introduction

The rapid advancement of Artificial Intelligence (AI) techniques has transformed numerous sectors, including healthcare, finance, cybersecurity, and logistics. However, this progress has also highlighted an important limitation: the lack of transparency of many of these models, particularly those regarded as black-box systems [1]. In many real-world contexts, obtaining accurate predictions is not sufficient; it is also essential to understand how and why a model makes certain decisions, especially when these decisions must be interpretable, auditable, or justifiable [2].

In this context, Explainable Artificial Intelligence (XAI) [3] aims to provide understandable explanations of model behaviour. Among the different approaches proposed in the literature, local explanation methods such as SHAP [4], LIME [5], *Integrated Gradients* [6], and *BreakDown* [7] have gained considerable relevance because they make it possible to analyse the contribution of input variables to specific decisions, supporting tasks such as model validation, bias detection, and increased trust in AI systems.

Despite the significant development of XAI techniques, evaluating the quality of explanations remains an open and complex problem [2]. Unlike the evaluation of models' predictive performance, for which well-established and widely accepted metrics exist, XAI does not always provide a reference explanation that can be used to directly determine the quality of an explanation. Consequently, numerous approaches and metrics have been proposed to evaluate different properties, such as fidelity, robustness, sensitivity, interpretability, and complexity [8, 9]. However, existing metrics are often implemented using *ad hoc* approaches, making them difficult to configure, compare, and reproduce across different experimental settings [10, 11]. Therefore, a unified and extensible software framework is needed to facilitate the consistent and reproducible evaluation of XAI methods.

To address these limitations, this work presents a Python library that implements 17 evaluation metrics and enables the assessment of explanations generated by different XAI methods across multiple models and datasets. The tool follows a modular and extensible architecture, allows experiments to be defined through `YAML` files, and generates comparative reports in `CSV` and `JSON` formats.

CHAPTER

2

Getting and running the library

This chapter describes the steps required to download, install, and verify the **XAIMetrics** library.

XAIMetrics is distributed as a Python package through its GitHub repository, available at `https://github.com/kdis-lab/XAIMetrics`. The current version of the package requires Python `3.13` or later.

Table 2.1 lists the direct dependencies declared by XAIMetrics and their minimum required versions.

Dependency	Minimum version
aix360	`>=0.3.0`
dalex	`>=1.7.2`
lime	`>=0.2.0.1`
numpy	`>=2.4.6`
pandas	`>=3.0.3`
pyod	`>=3.6.0`
PyYAML	`>=6.0.3`
quantus	`>=0.6.0`
shap	`>=0.52.0`
torch	`>=2.12.0`
pytest	`>=9.1.1`
Sphinx	`>=9.1.0`
sphinx-rtd-theme	`>=3.1.0`

Table 2.1: Minimum versions of the direct dependencies declared by XAIMetrics

The versions shown in Table 2.1 are lower bounds rather than exact versions. When the package is installed, `pip` automatically resolves these requirements and installs any additional transitive dependencies. In particular, packages such as `scikit-learn`, `joblib`, `matplotlib`, and `cloudpickle` may be installed as dependencies of the libraries listed above.

Some of the datasets, attribution files, and serialized models included in the repository are managed using Git Large File Storage (Git LFS). Therefore, both Git and Git LFS must be installed before cloning the repository. Git LFS only needs to be initialized once for each user account:

```
$ git lfs install
```

The library is distributed via GitHub, available at https://github.com/kdis-lab/XAIMetrics. The repository contains the following main folders:

- `docs`. Contains the documentation resources, including the user manual, the API documentation in PDF format, and the Sphinx project used to generate the HTML API reference.
- `examples`. Contains a basic usage notebook, an example YAML configuration file, sample datasets, pretrained models, attribution files, specific usage examples, and generated reports in CSV and JSON formats.
- `experiments`. Contains the datasets, trained models, attribution files, generated reports, and the Python script used to execute the experimental study.
- `tests`. Contains the automated tests used to verify the correct behaviour of the library and its components.
- `xai_metrics`. Contains the Python source code of the library, including the metric implementations, explainers, configuration management, experiment runner, and reporting functionality.

The library con be installed directly from the repository in editable mode:

```
$ git clone https://github.com/kdis-lab/XAIMetrics.git
$ cd XAIMetrics
$ pip install -e .
```

CHAPTER

3

XAIMetrics library

`XAIMetrics` is a Python library that provides a unified framework for generating and evaluating explanations produced by XAI methods. The library enables different explanation techniques to be applied and assessed across multiple models and datasets under consistent experimental conditions, facilitating systematic and reproducible comparisons between XAI methods.

`XAIMetrics` has been designed as a modular, extensible, and reproducible environment that integrates explanation generation, metric evaluation, experiment configuration, automated execution, and report generation. Experiments can be defined through declarative `YAML` configuration files, allowing users to specify datasets, models, explanation methods, evaluation metrics, and their corresponding parameters without modifying the source code of the library.

The main features of the library are as follows:

- Provides implementations of 17 metrics for evaluating different properties of feature-attribution explanations.
- Includes implementations of four local explanation methods: LIME, SHAP, BreakDown, and MAPLE.
- Supports both the generation of feature attributions and the evaluation of previously generated attribution files.
- Automatically discovers and evaluates the available combinations of datasets, predictive models, and XAI methods.
- Allows multiple explanation methods and evaluation metrics to be executed from the same experimental configuration.
- Uses declarative `YAML` files to define and reuse complete experiments, including the input paths, execution device, explanation methods, metrics, and their parameters.
- Supports both automatically constructed evaluation contexts and contexts provided directly at runtime.
- Uses automatic registration and discovery mechanisms for metrics and explanation methods, facilitating the integration of new components.

- Allows runtime dependencies to be supplied externally, including custom model-loading functions and explanation functions required by particular metrics.
- Records metrics or explanation methods that cannot be applied to a particular context without interrupting the remaining experiment.
- Generates an aggregated comparative report containing the dataset, model, metric, metric parameters, and results obtained for each XAI method.
- Generates observation-level reports for metrics whose outputs can be associated with the individual observations evaluated.
- Exports the aggregated results in `CSV` and `JSON` formats and the observation-level results in `CSV` format, facilitating their filtering, analysis, and reuse in other environments.
- Saves generated attribution matrices as `CSV` files that can subsequently be used by the evaluation workflow.

This chapter describes the architecture and main functionality of the library. It also presents the complete workflow supported by `XAIMetrics`, including experiment configuration, explanation generation, context construction, metric execution, and report generation.

3.1 XAIMetrics library architecture

The library has been developed as an open-source Python project. It follows a modular architecture that separates the responsibilities associated with configuration management, explanation generation, metric evaluation, experiment execution, and result reporting. This organization allows new metrics, explanation methods, and supporting components to be incorporated without modifying the main execution workflow.

The library is organized into six main modules: `base`, `config`, `explainers`, `metrics`, `runner`, and `reporting`. The main functionality of each module is described below:

- *base.* Contains the common abstractions, data structures, and registration mechanisms used throughout the library. It defines the `BaseMetric` and `BaseExplainer` classes, from which all metrics and explanation methods inherit, respectively. The module also defines two context classes. The `MetricContext` class groups the information required to evaluate a metric, including the predictive model, test data, labels, selected observations, attribution values, and execution device. The `ExplainerContext` class contains the information required to generate explanations, including the background data, the predictive model, the observations to explain, their associated targets, and the execution device. In addition, this module provides separate registries for metrics and explainers, together with the `register_metric` and `register_explainer` decorators. These mechanisms allow implementations to be discovered and instantiated automatically from their names. The `MetricSkipped` and `ExplainerSkipped` exceptions are also defined in this module and are used to indicate that a component cannot be applied to a particular experimental context without stopping the complete execution.
- *config.* Manages the loading and interpretation of the experimental configuration through the `ConfigController` class. A configuration can be supplied as a Python mapping or as the path to a `YAML` file. This module supports two ways of defining an experiment. A single context can be specified explicitly by providing direct paths to the model, data, labels, and attribution files. Alternatively, the library can discover multiple experimental contexts automatically from the configured dataset, model, and attribution directories. The module is also

responsible for loading the required data and models and for constructing the `MetricContext` and `ExplainerContext` objects used by the execution workflows. A custom model-loading function can be provided when the default loading mechanism is not suitable for a particular model format.

- *explainers*. Contains the implementations of the explanation methods integrated into the library. The current version provides LIME, SHAP, BreakDown, and MAPLE. Each explanation method is implemented as a class that inherits from `BaseExplainer` and implements the `explain()` method. This method receives a predictive model and a batch of observations and returns the corresponding feature-attribution matrix. The module also provides an automatic discovery mechanism that imports and registers the available explanation methods. Consequently, explainers can be selected and configured by name in the `YAML` file. New explanation techniques can be incorporated by implementing the common interface and registering the resulting class.

- *metrics*. Contains the implementations of the 17 evaluation metrics included in the library. Each metric is implemented as a class that inherits from `BaseMetric`, receives a `MetricContext`, and defines its computation through the `run()` method. Metric implementations are independent of the main execution workflow and are discovered automatically through the metric registry. This design allows users to select metrics by name in the experimental configuration and facilitates the addition of new evaluation measures. The metrics are organized into submodules according to the main explainability property they evaluate:

  - *metrics.complexity*. Contains metrics that evaluate the complexity and sparsity of an explanation.
  - *metrics.faithfulness*. Contains metrics that evaluate the correspondence between feature attributions and the behaviour of the predictive model.
  - *metrics.fidelity*. Contains metrics related to the fidelity of an explanation. This module is further divided into *completeness* and *soundness* submodules.
  - *metrics.fidelity.completeness*. Contains metrics that evaluate whether an explanation captures the information required to represent the model prediction sufficiently.
  - *metrics.fidelity.soundness*. Contains metrics that evaluate whether the information identified as relevant by an explanation is supported by the behaviour of the model.
  - *metrics.robustness*. Contains metrics that evaluate the stability of explanations under small changes in the input or model output.
  - *metrics.sensitivity*. Contains metrics that evaluate how strongly explanations change when the corresponding input observations are perturbed.

- *runner*. Coordinates the main execution workflows of the library. It provides the `run_explanation()` and `run_evaluation()` functions. The `run_explanation()` function manages the explanation-generation workflow. It discovers the registered explainers, loads the experimental configuration, constructs the required `ExplainerContext` objects, selects the configured explanation methods, and generates their attribution matrices. The generated attributions can be returned directly and optionally exported as `CSV` files. The `run_evaluation()` function manages the metric-evaluation workflow. It discovers the registered metrics, constructs or receives the required `MetricContext` objects, selects the configured metrics, injects the available runtime dependencies, and executes each metric for every valid combination of dataset, model, and XAI method. Both workflows record components that cannot be executed in the current context instead of interrupting the complete experiment. The evaluation workflow collects the results and forwards them to the reporting module to construct the aggregated and observation-level reports.

- *reporting*. Organizes, aggregates, and exports the outputs produced by the explanation and evaluation workflows. For the explanation workflow, this module converts the generated

attribution matrices into data frames and saves them as `CSV` files. The files are organized according to the corresponding dataset, model, and XAI method. For the evaluation workflow, the module generates two types of reports. First, it builds an aggregated comparative report in which each row identifies a dataset, model, metric, and metric-parameter configuration, while the results of the different XAI methods are represented in separate columns. When a metric returns multiple numeric values, these values are aggregated to produce the corresponding summary result. Second, the module builds observation-level reports for metric outputs that can be aligned with the evaluated observations. These reports preserve the individual metric value obtained for each observation and organize the results into columns corresponding to the evaluated XAI methods. The aggregated results from all datasets and models are combined into a single report and exported in both `CSV` and `JSON` formats. Observation-level reports are exported as separate `CSV` files for each metric, dataset, and model.

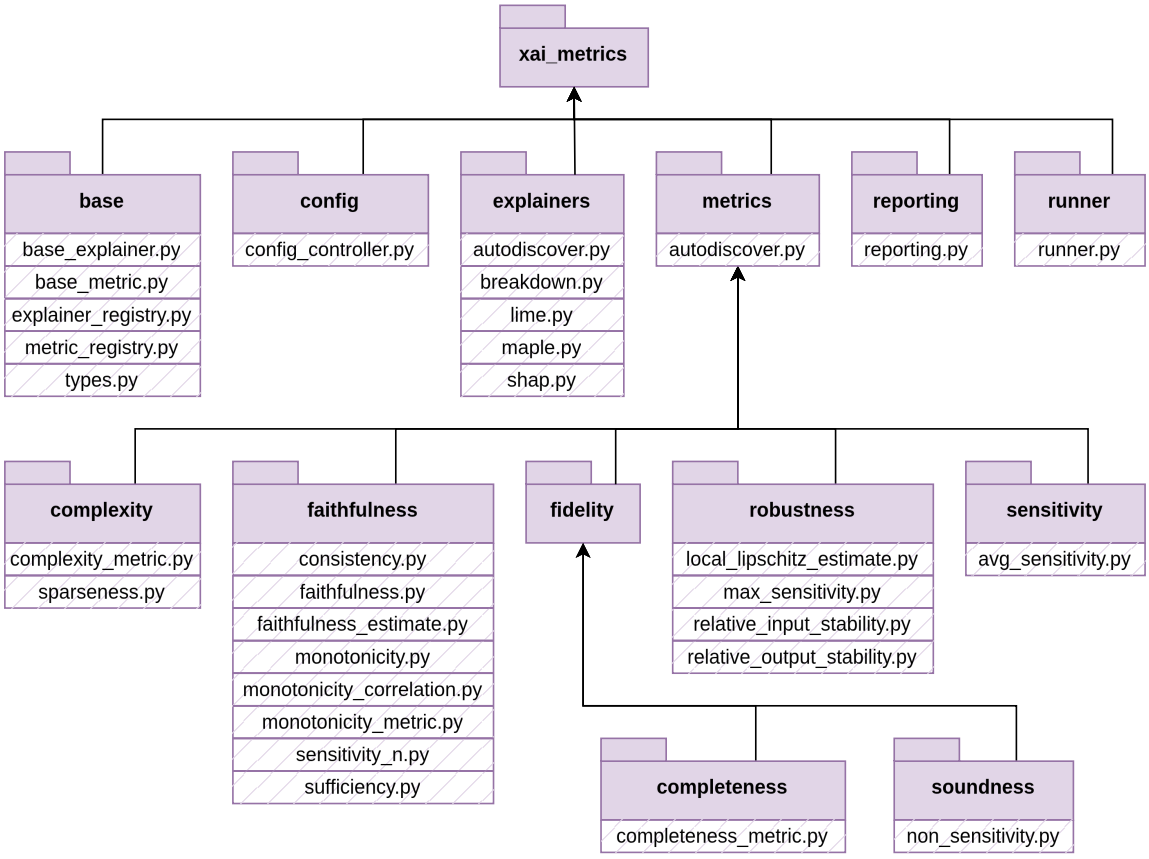

Figure 3.1: Architecture XAIMetrics library

Figure 3.1 illustrates the overall architecture of the **XAIMetrics** library and the relationships between the components involved in experiment configuration, explanation generation, metric evaluation, and report generation. The configuration module constructs the contexts required by the execution workflows, while the runner coordinates the registered explainers and metrics. Finally, the reporting module organizes and exports the generated attributions and evaluation results.

CHAPTER

4

Run available Metrics in the Library

In this chapter, the individual metrics implemented in the **XAIMetrics** library are presented. For each metric, a brief description of its purpose and the explainability property it evaluates is provided, followed by two illustrative examples. The first example shows how to execute the metric individually using its corresponding Python class, whereas the second demonstrates how to include it in an automated evaluation experiment configured through a `YAML` file and executed by the `runner` module

The metrics are organized according to the objective evaluation categories introduced in the proposed taxonomy and currently supported by the library. These include Faithfulness in Section 4.1, Sensitivity in Section 4.3, Robustness in Section 4.4, Complexity in Section 4.2, and Fidelity 4.5. The latter is further divided into Completeness in Section 4.5.1 and Soundness in Section 4.5.2.

4.1 Faithfulness

This category includes metrics that evaluate the extent to which an explanation reflects the predictive behaviour of the model in terms of feature contributions. For attribution-based explanations, these metrics analyse whether the features identified as the most relevant are also those that have the greatest influence on the model output. Therefore, they assess the relationship between the importance assigned by the explanation and the actual effect of each feature on the prediction.

Faithfulness metrics are particularly useful for determining whether an explanation accurately represents the decision-making process of the model. They make it possible to verify whether modifying, removing, or introducing the most relevant features produces the expected changes in the model output. In this way, they provide a quantitative assessment of whether the generated attributions are consistent with the model behaviour.

However, the results of these metrics may depend on the perturbation strategy, the feature ordering, and the presence of dependencies between input variables. Modifying features independently may generate unrealistic samples or fail to preserve the relationships present in the original data. Consequently, faithfulness scores should be interpreted while considering the characteristics of the dataset and the experimental configuration used to evaluate the explanations.

4.1.1 Consistency

The Consistency metric evaluates whether observations with similar explanations are assigned to the same predicted class [12]. To perform this comparison, the continuous attribution vectors are first transformed into discrete representations. For each observation, the metric computes the proportion of the remaining observations in its group that receive the same predicted class.

A high consistency score indicates that observations with equivalent attribution patterns frequently receive the same model prediction. Conversely, a low score indicates that similar explanations may be associated with different predicted classes. Therefore, the metric assesses whether the explanations consistently represent the decision-making behaviour of the model.

The parameters used by this metric can be summarized as follows:

- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `True`)
- `normalise (bool)`: whether to normalise the attribution values before discretising and comparing the explanations. (*Default:* `True`)

The implementation uses the default discretisation and normalisation functions provided by `Quantus`. In particular, the default discretisation function is `top_n_sign`, which converts each continuous attribution vector into a discrete representation according to the most relevant features and the signs of their attributions.

Before computing the metric, the model is set to evaluation mode, and the observations specified in the `MetricContext` are selected together with their labels and attribution values. The metric returns a Consistency score for each evaluated observation. When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `Consistency` class, placed in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.1.1), or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.1.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. The configuration also specifies that the original signs of the attribution values must be preserved by setting `abs` to `false`, while attribution normalisation remains enabled.

Examples of these usage modes are available in the library under: `examples/specific_examples/metrics/Consistency_examples.py`.

4.1.1.1 Direct evaluation using the Consistency class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Consistency` class is instantiated directly using the desired parameters. Calling the `run()` method returns one Consistency score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```python
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import Consistency

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = Consistency(
    context=context,
    params={
        "abs": False,
        "normalise": True
    }
)
scores = metric.run()
```

4.1.1.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Consistency"` metric, although this argument would not be necessary if `Consistency` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Consistency"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Consistency
```

```yaml
params:
  abs: false
  normalise: true
```

4.1.2 Faithfulness Estimate

The Faithfulness Estimate metric evaluates whether the importance assigned to the input features by an explanation is consistent with their actual influence on the model output [11]. To perform this evaluation, features are perturbed in groups according to their attribution importance. For each perturbation step, the metric compared the sum of the attribution values of the perturbed features with the corresponding decrease in the target model output.

The agreement between these two sequences is measured using the Pearson correlation coefficient. A high positive score indicates that features with larger attribution values also produce larger changes in the model output when perturbed. Conversely, a low or negative score indicates weak or inverse agreement between the feature importance assigned by the explanation and the behaviour of the model. Therefore, the metric assesses whether the attribution values accurately reflect the influence of the features on the prediction.

The parameters used by this metric can be summarized as follows:

- `features_in_step (int)`: number of features perturbed together at each evaluation step. (*Default:* `1`)
- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)
- `normalise (bool)`: whether to normalise the attribution values before computing the metric. (*Default:* `True`)
- `perturb_baseline (str)`: baseline used to replace the selected features during perturbation. The available values depend on the perturbation function provided by `Quantus` and commonly include `"black"`, `"white"`, `"mean"`, `"random"`, and `"uniform"`. (*Default:* `"black"`)

The implementation uses the default normalisation and perturbation functions provided by `Quantus`. The similarity between the attribution sums and the corresponding decreases in the model output is calculated using an internal safe implementation of the Pearson correlation coefficient. When either of the compared vectors has zero variance, this function returns 0.0 instead of an undefined `NaN` value.

Before computing the metric, the model is set to evaluation mode, and the observations specified in the `MetricContext` are selected together with their labels and attribution values. The metric returns one Faithfulness Estimate score for each evaluated observation. Higher values indicate stronger agreement between the importance assigned to the features and the changes produced in the model output after perturbation. When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used, and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and id not evaluated for that attribution configuration.

The metric is implemented through the `FaithfulnessEstimate` class in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.2.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.2.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. The configuration perturbs one feature at each step, preserves the original signs of the attribution normalisation, and uses the mean feature value as the perturbation baseline.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/FaithfulnessEstimate_examples.py`

4.1.2.1 Direct evaluation using the FaithfulnessEstimate class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `FaithfulnessEstimate` class is instantiated directly using the desired parameters. Calling the `run()` method returns one faithfulness estimate score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import FaithfulnessEstimate

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = FaithfulnessEstimate(
    context=context,
    params={
        "features_in_step": 1,
        "abs": False,
        "normalise": True,
        "perturb_baseline": "mean"
    }
)
scores = metric.run()
```

4.1.2.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"FaithfulnessEstimate"` metric, although this argument would not be necessary if `FaithfulnessEstimate` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["FaithfulnessEstimate"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: FaithfulnessEstimate
    params:
      features_in_step: 1
      abs: false
      normalise: true
      perturb_baseline: mean
```

4.1.3 Faithfulness

The Faithfulness metric evaluates whether the attribution values assigned to the input features reflect their actual influence on the model prediction [11]. For each evaluated observation, the metric first determines the class predicted by the model. It then replaces each feature individually with its corresponding baseline value and records the probability assigned to the originally predicted class after each replacement.

The score is computed as the negative Pearson correlation between the attribution values and the predicted-class probabilities obtained after replacing the individual features. Features with larger attribution values are expected to produce larger decreases in the predicted-class probability when replaced by their baseline values. Therefore, a high positive score indicates strong agreement between the importance assigned by the explanation and the influence of the features on the model prediction. Conversely, a low or negative score indicates weak or inverse agreement.

The parameters used by this metric can be summarized as follows:

- `base_values (array-like, optional)`: explicit baseline values used to replace the input features. The provided array must contain one baseline value for each feature. When specified, these values take priority over `base_strategy`. (*Default:* `None`)
- `base_strategy (str)`: strategy used to calculate a common baseline vector from the complete test dataset when `base_values` is not provided. The supported strategies are:
  - `"mean"`: uses the feature-wise mean of the test dataset;
  - `"median"`: uses the feature-wise median of the test dataset;
  - `"zero"`: uses a vector containing one zero for each feature.

  (*Default:* `"mean"`)

Before computing the metric, a common baseline vector is obtained either from the values explicitly provided through `base_values` or from the complete test dataset using the selected `base_strategy`. The observations specified in the `MetricContext` are then selected and evaluated independently together with their corresponding attribution vectors.

The metric returns one Faithfulness score for each evaluated observation. Higher positive values indicate stronger agreement between the attribution values and the decrease in predicted-class probability caused by replacing individual features. If the attribution vector or the probabilities obtained from the perturbed inputs have zero variance, the Pearson correlation is undefined, and the underlying `AIX360` implementation may return `NaN`.

If an unsupported value is provided through `base_strategy`, the metric raises a `ValueError`. Moreover, the implementation relies on the `faithfulness_metric` function provided by `AIX360`, which required a classification model exposing a `predict_proba` method. If the evaluated model does not implement this method, the metric raises an `AttributeError`.

The metric is implemented through the `Faithfulness` class, placed in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.3.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.3.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. The baseline vector is calculated using the feature-wise mean of the complete test dataset. This experiment assumes that the loaded model provides the `predict_proba` method required by the metric.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/Faithfulness_examples.py`

4.1.3.1 **Direct evaluation using the Faithfulness class**

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Faithfulness` class is instantiated directly using the desired parameters. Calling the `run()` method returns one faithfulness score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import Faithfulness

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = Faithfulness(
    context=context,
    params={
        "base_strategy": "mean"
    }
)
scores = metric.run()
```

4.1.3.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Faithfulness"` metric, although this argument would not be necessary if `Faithfulness` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Faithfulness"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Faithfulness
    params:
      base_strategy: mean
```

4.1.4 Monotonicity Correlation

The Monotonicity Correlation metric evaluates whether feature attribution values are monotonically related to the uncertainty produced in the model output when the corresponding features are perturbed [13]. For each evaluated observation, the features are ordered according to their attribution values and divided into groups. Each group is perturbed repeatedly, and the uncertainty associated with that group is estimated from the changes produced in the target model output.

The metric compares the sum of the attribution values in each feature group with the corresponding uncertainty estimate using the Spearman rank correlation coefficient. A high positive score indicates that groups containing features with greater attribution values tend to produce greater uncertainty in the model output when perturbed. Conversely, a low or negative score indicates a weak or inverse monotonic relationship between the importance assigned by the explanation and the effect of perturbing the corresponding features. Therefore, the metric assesses whether the ordering induced by the attribution values agrees with the sensitivity of the model output to feature perturbations.

The parameters used by this metric can be summarized as follows:

- `eps (float)`: threshold used when calculating the inverse prediction factor employed to scale the changes in the model output. This parameter avoids numerical instability when the magnitude of the original output is close to zero. (*Default:* `1e-5`)
- `nr_samples (int)`: number of perturbed samples generated for each feature group to estimate the uncertainty associated with its perturbation. (*Default:* `100`)
- `features_in_step (int)`: number of features included in each group and perturbed together at every evaluation step. (*Default:* `1`)
- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `True`)
- `normalise (bool)`: whether to normalise the attribution values before ordering the features and computing the metric. (*Default:* `True`)
- `perturb_baseline (str)`: baseline used when perturbing the selected features. The available values depend on the perturbation function provided by `Quantus` and commonly include `"black"`, `"white"`, `"mean"`, `"random"`, and `"uniform"`. (*Default:* `"uniform"`)

The implementation uses the default normalisation and perturbation functions provided by `Quantus`. The relationship between the attribution sums and the uncertainty estimates is calculated using an internal safe implementation of the Spearman correlation coefficient. When either of the compared vectors has zero variance, this function returns 0.0 instead of an undefined `NaN` value.

Before computing the metric, the model is set to evaluation mode, and the observations specified in the `MetricContext` are selected together with their target labels and attribution values. For each observation, the feature groups are perturbed `nr_samples` times. The uncertainty associated with each group is estimated from the mean squared change in the target model output, scaled relative to the magnitude of the original output. The metric then returns one Monotonicity Correlation score for each evaluated observation.

Higher values indicate a stronger positive monotonic relationship between feature importance and the uncertainty caused by perturbing the corresponding feature groups. When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `MonotonicityCorrelation` class in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.4.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.4.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. The configuration perturbs one feature at each step, generates 100 perturbation samples for each feature group, uses the absolute and normalised attribution values, and replaces the perturbed features with their mean values.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/MonotonicityCorrelation_examples.py`

4.1.4.1 Direct evaluation using the MonotonicityCorrelation class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `MonotonicityCorrelation` class is instantiated directly using the desired parameters. Calling the `run()` method returns one monotonicity correlation score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```python
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import MonotonicityCorrelation

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = MonotonicityCorrelation(
    context=context,
    params={
        "eps": 1e-5,
        "nr_samples": 100,
        "features_in_step": 1,
        "abs": True,
        "normalise": True,
        "perturb_baseline": "mean"
    }
)
scores = metric.run()
```

4.1.4.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"MonotonicityCorrelation"` metric, although this argument would not be necessary if `MonotonicityCorrelation` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["MonotonicityCorrelation"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
```

```yaml
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: MonotonicityCorrelation
    params:
      eps: 1e-5
      nr_samples: 100
      features_in_step: 1
      abs: true
      normalise: true
      perturb_baseline: mean
```

4.1.5 Monotonicity Metric

The Monotonicity Metric evaluates whether the probability assigned to the originally predicted class increases monotonically as the input features are progressively restored from a baseline vector [14]. For each evaluated observation, the metric first obtains the class predicted by the model using the complete original input. It then creates an initial input containing the baseline value of each feature and progressively restores the original feature values in increasing order of their attribution values.

After restoring each feature, the metric records the probability assigned to the originally predicted class. The resulting sequence is expected to be monotonically non-decreasing, meaning that restoring progressively more relevant information should not reduce the probability of the original prediction. The metric returns `True` when this condition is satisfied and `False` otherwise.

The parameters used by this metric can be summarized as follows:

- `base_values (array-like, optional)`: explicit baseline values used as the initial feature values. The provided array must contain one baseline value for each input feature. When specified, these values take priority over `base_strategy`. (*Default:* `None`)
- `base_strategy (str)`: strategy used to calculate a common baseline vector from the complete test dataset when `base_values` is not provided. The supported strategies are:
  - `"mean"`: uses the feature-wise mean of the test dataset;
  - `"median"`: uses the feature-wise median of the test dataset;
  - `"zero"`: uses a vector containing one zero for each feature.

  (*Default:* `"mean"`)

Before computing the metric, a common baseline vector is obtained either from the values explicitly provided through `base_values` or from the complete test dataset using the selected `base_strategy`. The observations specified in the `MetricContext` are then selected and evaluated independently together with their corresponding attribution vectors.

For each observation, the features are ordered by increasing attribution value. Starting from the baseline vector, their original values are restored cumulatively following this order. After each restoration, the probability assigned to the class predicted for the original complete input is obtained.

The metric returns one Boolean result for each evaluated observation. A value of `True` indicates that the predicted-class probability never decreases as the features are progressively restored, whereas `False` indicates that the monotonicity condition is violated. When the Boolean results

are averaged, `True` and `False` are interpreted as (1) and (0), respectively. Therefore, the resulting mean represents the proportion of evaluated observations that satisfy the monotonicity criterion.

If an unsupported value is provided through `base_strategy`, the metric raises a `ValueError`. Moreover, the implementation relies on the `monotonicity_metric` function provided by `AIX360`, which requires a classification model exposing a `predict_proba` method. If the evaluated model does not implement this method, the metric raises an `AttributeError`.

The metric is implemented through the `MonotonicityMetric` class in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.5.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.5.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. The initial baseline vector is calculated using the feature-wise mean of the complete test dataset. The experiment assumes that the loaded model provides the `predict_proba` method required by the metric.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/MonotonicityMetric_examples.py`

4.1.5.1 Direct evaluation using the MonotonicityMetric class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `MonotonicityMetric` class is instantiated directly using the desired parameters. Calling the `run()` method returns one monotonicity metric score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import MonotonicityMetric

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = MonotonicityMetric(
    context=context,
    params={
        "base_strategy": "mean"
    }
)
scores = metric.run()
```

4.1.5.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"MonotonicityMetric"` metric, although this argument would not be necessary if `MonotonicityMetric` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["MonotonicityMetric"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: MonotonicityMetric
    params:
      base_strategy: mean
```

4.1.6 Monotonicity

The Monotonicity metric evaluates whether the target model output evolves monotonically as feature groups are progressively introduced according to their attribution values [14]. For each evaluated observation, the features are ordered by increasing attribution value. The evaluation starts from an input defined by a baseline, and the original feature values are progressively introduced in groups following this ordering.

After introducing each feature group, the metric evaluates the model output associated with the target label. The resulting sequence is expected to be monotonically non-decreasing, meaning that adding progressively more relevant information should not reduce the target model output. The metric returns `True` when this condition is satisfied throughout the complete sequence and `False` when the output decreases in at least one step.

The parameters used by this metric can be summarized as follows:

- `features_in_step (int)`: number of features introduced together at each evaluation step. (*Default:* `1`)
- `abs (bool)`: whether to use the absolute values of the attributions before ordering the features and computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `True`)
- `normalise (bool)`: whether to normalise the attribution values before ordering the features and computing the metric. (*Default:* `True`)

- `perturb_baseline (str)`: baseline used to initialise the input before the original feature values are progressively introduced. The available values depend on the perturbation function provided by `Quantus` and commonly include `"black"`, `"white"`, `"mean"`, `"random"`, and `"uniform"`. (*Default:* `"black"`)

The implementation uses the default normalisation and perturbation functions provided by `Quantus`. For each observation, the input is initially replaced by the selected baseline. The original feature values are then introduced in groups of size `features_in_step`, following the increasing order of their attribution values.

Before computing the metric, the model is set to evaluation mode, and the observations specified in the `MetricContext` are selected together with their target labels and attribution values. After each feature group is introduced, the target model output is evaluated and compared with the output obtained in the previous step.

The metric returns one Boolean value for each evaluated observation. A value of `True` indicates that the target model output never decreases as the feature groups are progressively introduced, whereas `False` indicates that the monotonicity condition is violated at least once. When the Boolean results are averaged, `True` and `False` are interpreted as (1) and (0), respectively. Therefore, the resulting mean represents the proportion of observations that satisfy the monotonicity criterion.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `Monotonicity` class, placed in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.6.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.6.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. The configuration introduces one feature at each step, uses absolute and normalised attribution values, and initialises the perturbed input using the `"black"` baseline.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/Monotonicity_examples.py`

4.1.6.1 Direct evaluation using the Monotonicity class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Monotonicity` class is instantiated directly using the desired parameters. Calling the `run()` method returns one monotonicity metric score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import Monotonicity

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = Monotonicity(
    context=context,
    params={
```

```
        "features_in_step": 1,
        "abs": True,
        "normalise": True,
        "perturb_baseline": "black"
    }
)
scores = metric.run()
```

4.1.6.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Monotonicity"` metric, although this argument would not be necessary if `Monotonicity` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Monotonicity"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Monotonicity
    params:
      features_in_step: 1
      abs: true
      normalise: true
      perturb_baseline: black
```

4.1.7 Sensitivity-N

The Sensitivity-N metric evaluates whether the attribution values assigned to groups of input features agree with the changes produced in the target model output when those features are perturbed [14]. For each evaluated observation, the features are ordered by decreasing attribution value and progressively perturbed in groups. At each step, the metric records both the attribution

values associated with the processed feature group and the corresponding variation in the target model output.

The agreement between these quantities is measured using the Pearson correlation coefficient. A high positive correlation indicates that feature groups with greater attribution values tend to produce larger changes in the target output when perturbed. Conversely, a low or negative correlation indicates weak or inverse agreement between the importance assigned by the explanation and the actual influence of the corresponding features on the model prediction. Therefore, the metric assesses whether the attribution values accurately represent the sensitivity of the model output to feature perturbations.

The parameters used by this metric can be summarized as follows:

- `n_max_percentage (float)`: maximum proportion of input features considered during the evaluation. For example, a value of `0.8` limits the experiment to perturbation steps covering up to 80% of the available features. (*Default:* `0.8`)
- `features_in_step (int)`: number of features perturbed together at each evaluation step. (*Default:* `1`)
- `abs (bool)`: whether to use the absolute values of the attributions before ordering the features and computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)
- `normalise (bool)`: whether to normalise the attribution values before computing the metric. (*Default:* `True`)
- `perturb_baseline (str)`: baseline used to replace the selected features during perturbation. The available values depend on the perturbation function provided by `Quantus` and commonly include `"black"`, `"white"`, `"mean"`, `"random"`, and `"uniform"`. (*Default:* `"black"`)

The implementation uses the default Pearson correlation, normalisation function, and baseline-replacement function provided by `Quantus`. For each observation, the features are ordered by decreasing attribution value and processed in groups of size `features_in_step`. Only the perturbation steps covered by `n_max_percentage` are included in the evaluation.

Before computing the metric, the model is set to evaluation mode, and the observations specified in the `MetricContext` are selected together with their target labels and attribution values. After each feature group is perturbed, the variation in the target model output is compared with the attribution values associated with that group.

The metric returns the Sensitivity-N result produced by `Quantus`. Higher values indicate stronger agreement between the attribution values and the changes caused in the target model output by perturbing the corresponding features. Since result aggregation is enabled by default in the underlying `Quantus` implementation, the returned values may represent an aggregation of the correlations calculated across the evaluated perturbation steps rather than one independent score for every input observation.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `SensitivityN` class, placed in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.7.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.7.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. In this example, all input features are considered, one feature is perturbed at each step, the original signs of the attribution values are preserved, normalisation is enabled, and the perturbed features are replaced using the `"uniform"` baseline.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/SensitivityN_examples.py`

4.1.7.1 Direct evaluation using the SensitivityN class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `SensitivityN` class is instantiated directly using the desired parameters. Calling the `run()` method returns one sensitivity-n score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import SensitivityN

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = SensitivityN(
    context=context,
    params={
        "n_max_percentage": 1.0,
        "features_in_step": 1,
        "abs": False,
        "normalise": True,
        "perturb_baseline": "uniform"
    }
)
scores = metric.run()
```

4.1.7.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"SensitivityN"` metric, although this argument would not be necessary if `SensitivityN` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["SensitivityN"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: SensitivityN
    params:
      n_max_percentage: 1.0
      features_in_step: 1
      abs: false
      normalise: true
      perturb_baseline: uniform
```

4.1.8 Sufficiency

The Sufficiency metric evaluates whether observations with similar explanations receive the same model prediction [12]. To perform this evaluation, the attribution vectors of all observations included in the evaluation batch are flattened and compared using a pairwise distance function. The resulting distance matrix is normalised, and two explanations are considered similar when the distance between them is less than or equal to a predefined threshold.

For each evaluated observation, the metric identifies the remaining observations whose explanations satisfy this similarity condition and calculates the proportion that receive the same predicted class. The observation itself is excluded from the comparison. A high score indicates that observations with similar explanations frequently exhibit the same model behaviour. Conversely, a low score indicates that similar attribution patterns may be associated with different predicted classes. If no other explanation lies within the specified distance threshold, the metric returns a score of `0.0` for that observation.

The parameters used by this metric can be summarized as follows:

- `threshold (float)`: maximum normalised distance between two attribution vectors for their explanations to be considered similar. Lower values impose a stricter similarity criterion, whereas higher values allow more distant explanations to be included in the comparison. (*Default:* `0.6`)
- `distance_func (str)`: distance function used to compare the attribution vectors. The value is passed to `Quantus` and is generally a valid distance name supported by SciPy, such as `"seuclidean"`. (*Default:* `"seuclidean"`)
- `abs (bool)`: whether to use the absolute values of the attributions before computing the distances between explanations. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `True`)
- `normalise (bool)`: whether to normalise the attribution values before comparing the explanations. (*Default:* `True`)

The implementation uses the default attribution normalisation function provided by `Quantus`. For the complete set of evaluated observations, the attribution vectors are flattened and their pairwise distances are calculated using `distance_func`. After normalising the distance matrix, the `threshold` parameter is used to determine which explanations are sufficiently similar.

Before computing the metric, the model is set to evaluation mode, and the observations specified in the `MetricContext` are selected together with their labels and attribution values. The model predictions are then compared among observations whose explanations satisfy the similarity criterion.

The metric returns one Sufficiency score for each evaluated observation. Scores range from `0.0` to `1.0`. Higher values indicate that observations with similar explanations more frequently receive the same predicted class. Since each explanation is compared only with the other observations included in the same evaluation batch, the resulting scores depend on the observations selected in the `MetricContext`. Changing the evaluated batch may therefore change the neighbourhood of each explanation and its resulting score.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `Sufficiency` class, placed in the module `xai_metrics.metrics.faithfulness`. It can be executed directly using this class (Section 4.1.8.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.1.8.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. In this example, all input features are considered, one feature is perturbed at each step, the original signs of the attribution values are preserved, normalisation is enabled, and the perturbed features are replaced using the `"uniform"` baseline.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/Sufficiency_examples.py`

4.1.8.1 **Direct evaluation using the Sufficiency class**

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Sufficiency` class is instantiated directly using the desired parameters. Calling the `run()` method returns one sufficiency score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.faithfulness import Sufficiency

config_path = "examples/specific_examples/config.yaml"
metric = Sufficiency(
    context=context,
    params={
        "threshold": 0.6,
        "distance_func": "seuclidean",
        "abs": True,
        "normalise": True
    }
)
```

4.1.8.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Sufficiency"` metric, although this argument would not be necessary if `Sufficiency` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Sufficiency"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Sufficiency
    params:
      threshold: 0.6
      distance_func: seuclidean
      abs: true
      normalise: true
```

4.2 Complexity

This category includes metrics that evaluate the extent to which an explanation is concise and concentrated on a limited number of input features or explanatory elements [15]. For attribution-based explanations, these metrics analyse how the attribution values are distributed across the input features. An explanation is considered less complex when most of its relevance is assigned to a small subset of features, whereas explanations that distribute importance across many features are considered more complex.

Complexity metrics are particularly useful for assessing whether an explanation can be represented in a compact and understandable form. Explanations involving fewer relevant features may be easier to inspect and interpret, especially in high-dimensional problems. These metrics can quantify properties such as the entropy or dispersion of the attribution distribution, as well as the degree to which feature importance is concentrated on a small number of variables.

However, a less complex explanation is not necessarily more faithful or informative. Excessively sparse explanations may omit features that contribute meaningfully to the model prediction, while

more distributed explanations may accurately reflect models whose decisions depend on multiple interacting variables. Consequently, complexity scores should be interpreted together with other evaluation properties and considering the characteristics of the model, dataset, and explanation method.

4.2.1 **Complexity**

The Complexity metric evaluates how widely the attribution importance is distributed across the input features [10]. For each evaluated observation, the absolute attribution values are converted into relative contributions by dividing the importance of each feature by the total attribution distribution.

An explanation whose importance is concentrated on a small number of features produces a low entropy value and is therefore considered less complex. Conversely, when the attribution importance is distributed more uniformly across many features, the entropy increases and the explanation is considered more complex. Thus, lower scores indicate more concise explanations, whereas higher scores indicate that a larger number of features contribute substantially to the explanation.

The parameters used by this metric can be summarized as follows:

- `normalise (bool)`: whether to normalise the attribution values before computing their relative contribution and entropy. When this parameter is enabled, the default normalisation function provided by `Quantus` is used. (*Default:* `True`)

The absolute-value operation is always applied to the attribution values before computing the metric. This behaviour is defined by the underlying `Quantus` implementation and is fixed in the wrapper since the `abs` parameter is not exposed through the metric configuration. In addition, if all the attribution values supplied to the wrapper are negative, they are explicitly converted to their absolute values before being passed to `Quantus`.

Before computing the metric, the model is set to training mode, and the observations specified in the `MetricContext` are selected together with their labels and attribution values. The metric returns one Complexity score for each evaluated observation. Lower values indicate that the attribution importance is concentrated on fewer features, while higher values indicate a more widely distributed attribution pattern. The mean of the returned scores can be used as an aggregated measure of explanation complexity for the evaluated observations.

The metric is implemented through the `Complexity` class, which can be found in the module `xai_metrics.metrics.complexity`. It can be executed directly using this class (Section 4.2.1.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.2.1.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. Attribution normalisation is enabled before calculating the entropy-based complexity score.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/Complexity_examples.py`

4.2.1.1 Direct evaluation using the Complexity class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Complexity` class is instantiated directly using the desired parameters. Calling the `run()` method returns one complexity score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```python
from xai_metrics.config import ConfigController
from xai_metrics.metrics.complexity import Complexity

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = Complexity(
    context=context,
    params={
        "normalise": True
    }
)
scores = metric.run()
```

4.2.1.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Complexity"` metric, although this argument would not be necessary if `Complexity` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Complexity"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Complexity
    params:
      normalise: true
```

4.2.2 **Sparseness**

The Sparseness metric evaluates how concentrated the attribution magnitude is across the input features [16]. For each evaluated observation, the absolute attribution values are sorted, and the Gini index of the resulting attribution distribution is calculated.

An explanation that assigns most of its attribution magnitude to a small subset of features produces a high Sparseness score and is therefore considered more concise. Conversely, when the attribution magnitude is distributed relatively uniformly across many features, the score decreases. Thus, higher values indicate that fewer features account for most of the explanation, whereas values close to (0) indicate a more uniform attribution distribution.

The parameters used by this metric can be summarized as follows:

- `normalise (bool)`: whether to normalise the attribution values before computing the Gini index. When this parameter is enabled, the default normalisation function provided by `Quantus` is used. (*Default:* `True`)

The absolute-value operation is always applied to the attribution values before computing the metric. This behaviour is defined by the underlying `Quantus` implementation and is fixed in the wrapper since the `abs` parameter is not exposed through the metric configuration. In addition, if all the attribution values supplied to the wrapper are negative, they are explicitly converted to their absolute values before being passed to `Quantus`.

The metric returns one Sparseness score for each evaluated observation. Higher values indicate that the attribution magnitude is concentrated on fewer features, while lower values indicate a more uniform distribution across the input features. The mean of the returned scores can be used as an aggregated measure of explanation sparseness for the evaluated observations.

The metric is implemented through the `Sparseness` class, which can be found in the module `xai_metrics.metrics.complexity`. It can be executed directly using this class (Section 4.2.2.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.2.2.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. Attribution normalisation is enabled before calculating the Gini-based Sparseness score.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/Sparseness_examples.py`

4.2.2.1 **Direct evaluation using the Sparseness class**

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Sparseness` class is instantiated directly using the desired parameters. Calling the `run()` method returns one sparseness score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```python
from xai_metrics.config import ConfigController
from xai_metrics.metrics.complexity import Sparseness

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = Sparseness(
    context=context,
    params={
        "normalise": True
    }
)
scores = metric.run()
```

4.2.2.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Sparseness"` metric, although this argument would not be necessary if `Sparseness` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Sparseness"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Sparseness
    params:
      normalise: true
```

4.3 Sensitivity

This category includes metrics that evaluate how much an explanation changes when small modifications are introduced into the input data, the model parameters, or the explanation-generation

process. For attribution-based explanations, these metrics analyse whether similar inputs produce similar attribution patterns and whether the explanation remains stable under minor or irrelevant perturbations.

Sensitivity metrics are particularly useful for identifying explanations that change excessively in response to small variations that do not substantially alter the model prediction. An explanation with low sensitivity is generally considered more stable, since minor perturbations produce only limited changes in the resulting attribution values. Conversely, high sensitivity indicates that small changes in the experimental conditions may lead to substantially different explanations.

However, sensitivity scores depend on the type, magnitude, and distribution of the perturbations applied during the evaluation. Perturbations that are too large may alter the semantic meaning of the input or change the model prediction, while perturbations that are too small may fail to reveal meaningful instability. The results may also depend on the number of perturbation samples and the distance function used to compare explanations. Consequently, sensitivity scores should be interpreted considering the perturbation strategy and the characteristics of the evaluated data and model.

4.3.1 **Average Sensitivity**

The Average Sensitivity metric evaluates how much an explanation changes, on average, when small random perturbations are applied to the corresponding input [17]. For each evaluated observation, several perturbed versions of the original input are generated, and their explanations are recomputed using an explanation function supplied by the user.

For each perturbation, the metric compared the new explanation with the original attribution vector. The sensitivity is calculated as the norm of the difference between both explanations divided by the norm of the original explanation. The final score is obtained by averaging these relative changes across all the generated perturbations.

A low Average Sensitivity score indicates that the explanation changes only slightly under small input perturbations and is therefore considered more stable. Conversely, a high score indicates that minor changes in the input produce substantially different explanations, suggesting greater sensitivity and lower robustness.

The parameters used by this metric can be summarized as follows:

- `nr_samples (int)`: number of perturbed inputs generated for each evaluated observation. A larger number of samples provides a more extensive estimate of sensitivity but increases the computational cost because the explanation must be recomputed for every perturbed input. (*Default:* `200`)
- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)
- `normalise (bool)`: whether to normalise the original and perturbed attribution values before comparing their explanations. When enabled, the default normalisation function provided by `Quantus` is used. (*Default:* `False`)
- `lower_bound (float)`: lower bound of the uniform noise used to generate the input perturbations. (*Default:* `0.2`)
- `upper_bound (float, optional)`: upper bound of the uniform noise used to generate the input perturbations. When no value is specified, the default behaviour defined by `Quantus` is used. (*Default:* `None`)

In addition to these parameters, the metric requires an external explanation function, provided through the `explain_func` argument. This funciton is necessary because the explanations must be recomputed after perturbing each input. It must be compatible with the interface expected by `Quantus` and use the following function signature:

```
def explain_func(model, inputs, targets=None, **kwargs):
...
```

The `model` argument will contain the model being evaluated, `inputs` will contain the observations whose explanations must be generated, and `targets` optionally will identify the target outputs. The function must return a `NumPy array` containing the generated attribution vectors, with one row for each input observation. Additional arguments may be received through `**kwargs` for compatibility with the `Quantus` interface. If no explanation function is provided, the constructor raises a `ValueError`.

The implementation uses the default comparison, norm, normalisation, and perturbation functions provided by `Quantus`. By default, explanations are compared using their element-wise difference, while the numerator and denominator of the relative change are calculated using the Frobenius norm. The perturbed inputs are generated by adding uniform noise within the configured bounds.

Before computing the metric, the model is set to training mode, following the current implementation of the wrapper. The observations specified in the `MetricContext` are selected together with their target labels and original attribution values. The execution device stored in the context is also passed to `Quantus`.

The metric returns one Average Sensitivity score for each evaluated observation. Lower values indicate that the generated explanations are more stable under small random input perturbations, whereas higher values indicate greater variation between the original and perturbed explanations.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `AvgSensitivity` class, placed in the module `xai_metrics.metrics.sensitivity`. It can be executed directly using this class (Section 4.3.1.2) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.3.1.3).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. For each observation, 20 perturbed inputs are generated using uniform noise with a lower bound of 0.2. The original signs of the attribution values are preserved, and attribution normalisation is disabled.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/AvgSensitivity_examples.py`

The explanation function used by these examples is available under: `examples/specific_examples/metrics/make_explain_func.py`

4.3.1.1 LIME explanation function

Some evaluation metrics require an explanation function that can recompute feature attributions for perturbed observations during their execution. In this example, the required function is created by `make_lime_explain_func()` using the `LIMEExplainer` implementation provided by `XAIMetrics`.

The function first constructs the corresponding `ExplainerContext` from the example configuration file. It then defines the parameters used by LIME and instantiates a `LIMEExplainer`. Finally, the `as_explain_func()` method converts the explainer into a callable that follows the interface expected by the evaluation metrics and by `Quantus`.

```
import pandas as pd
from pathlib import Path
from xai_metrics.config import ConfigController
from xai_metrics.explainers.lime import LIMEExplainer

config_path = "examples/specific_examples/config.yaml"

def make_lime_explain_func():
    context, _ = ConfigController(config_path).build_explainers_context()

    params = {
        "mode": "classification",
        "random_state": 42,
        "num_samples": 5000,
        "labels": [1],
        "distance_metric": "euclidean"
    }

    return LIMEExplainer(context=context, params=params).as_explain_func()
```

In this configuration, LIME operates in classification mode, uses a fixed random seed, generates 5000 perturbed samples, computes distances using the Euclidean metric, and produces the explanation associated with class 1.

This implementation avoids duplicating the internal explanation logic in the example. The returned callable delegates input preparation, model prediction, LIME execution, and attribution formatting to the explainer already integrated into `XAIMetrics`.

4.3.1.2 Direct evaluation using the AvgSensitivity class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `AvgSensitivity` class is instantiated with the desired parameters and the explanation function returned by `make_lime_explain_func()`. Calling the `run()` method returns one average sensitivity score for each evaluated observation, and the mean of these scores can be calculated as an aggregated result.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.config import ConfigController
from xai_metrics.metrics.sensitivity import AvgSensitivity

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = AvgSensitivity(
```

```
    context=context,
    params={
        "nr_samples": 20,
        "abs": False,
        "normalise": False,
        "lower_bound": 0.2
    },
    explain_func = make_lime_explain_func()
)
scores = metric.run()
```

4.3.1.3 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file.

The explanation function cannot be defined directly in the `YAML` file. Therefore, it is created in Python and passed to `run_evaluation()` through the `explain_func` dependency. The runner subsequently provides this function when instantiating metrics that require explanations to be recomputed.

The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"AvgSensitivity"` metric, although this argument would not be necessary if `AvgSensitivity` is the only metric defined in the configuration.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["AvgSensitivity"],
    config=config_path,
    report_output_dir=None,
    explain_func = make_lime_explain_func()
)
```

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

metrics:
  - name: AvgSensitivity
```

```
params:
  nr_samples: 20
  abs: false
  normalise: false
  lower_bound: 0.2
```

4.4 Robustness

This category includes metrics that evaluate the stability of explanations under small perturbations of the input, assuming that the corresponding model prediction remains approximately unchanged [15]. For attribution-based explanations, these metrics analyse whether similar observations produce similar attribution patterns and whether minor changes in the input lead to disproportionate variations in the generated explanations.

Robustness metrics are particularly useful for determining whether an explanation method produces stable and reliable results in local regions of the input space. A robust explanation should remain similar for observations that are close to each other and receive comparable model predictions. These metrics, therefore, make it possible to identify explanation methods whose outputs vary excessively despite limited changes in the input or model behaviour.

However, robustness scores may depend strongly on the perturbation strategy, the distance functions used to compare inputs and explanations, and the size of the neighborhood considered during the evaluations. Perturbations that are too large may alter the model prediction or generate unrealistic observations, while excessively small perturbations may not reveal meaningful instability. The results may also be affected by differences in feature scales and by dependencies between input variables. Consequently, robustness scores should be interpreted considering the characteristics of the dataset, the model, and the experimental configuration.

4.4.1 Local Lipschitz Estimate

The Local Lipschitz Estimate metric evaluates the local stability of an explanation by comparing changes in the explanation with changes in the corresponding input [11]. For each evaluated observation, several neighbour inputs are generated by applying small Gaussian perturbations. Their explanations are then recomputed using an explanation function provided by the user.

For each perturbed input, the metric calculates the ratio between the distance separating the original and perturbed explanations and the distance separating the corresponding inputs. The Local Lipschitz Estimate is the maximum ratio obtained across all sampled neighbours:

$$\max_{x' \in \mathcal{N}(x)} \frac{\|\Phi(x) - \Phi(x')\|}{\|x - x'\|},$$

where (x) is the original observation, (x') is a perturbed neighbour, and (Φ) represents the explanation function.

A low Local Lipschitz Estimate indicates that nearby inputs receive similar explanations and therefore suggests greater local robustness. Conversely, a high score indicates that small changes in the input may produce comparatively large changes in the explanation. Since the metric evaluates only a finite number of sampled neighbours, the resulting value is an approximation of the maximum local variation rather than the exact Lipschitz constant over the complete neighbourhood.

The parameters used by this metric can be summarized as follows:

- `nr_samples (int)`: number of neighbour inputs generated for each evaluated observation. Increasing this value provides a more extensive exploration of the local neighbourhood but also increases the computational cost, since an explanation must be generated for every perturbed input. (*Default:* `200`)
- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)
- `normalise (bool)`: whether to normalise the original and perturbed attribution values before comparing the explanations. When enabled, the default normalisation function provided by `Quantus` is used. (*Default:* `True`)
- `perturb_mean (float)`: mean of the Gaussian noise used to generate neighbouring inputs. (*Default:* `0.0`)
- `perturb_std (float)`: standard deviation of the Gaussian noise used to generate neighbour inputs. Larger values produce more distant neighbours and therefore define a wider local region around the original observation. (*Default:* `0.1`)

In addition to these parameters, the metric requires an external explanation function, provided through the `explain_func` argument. This function is necessary because the explanations must be recomputed after perturbing each input. It must be compatible with the interface expected by `Quantus` and use the following function signature:

```
1 def explain_func(model, inputs, targets=None, **kwargs):
2 ...
```

The `model` argument will contain the model being evaluated, `inputs` will contain the observations whose explanations must be generated, and `targets` optionally will identify the target outputs. The function must return a `NumPy array` containing the generated attribution vectors, with one row for each input observation. Additional arguments may be received through `**kwargs` for compatibility with the `Quantus` interface. If no explanation function is provided, the constructor raises a `ValueError`.

The implementation uses the default comparison, distance, normalisation, and perturbation functions provided by `Quantus`. The changes in both the inputs and the explanations are measured using the Euclidean distance, while neighbouring inputs are generated by adding Gaussian noise with the configured mean and standard deviation.

Before computing the metric, the model is set to training mode, following the current implementation of the wrapper. The observations specified in the `MetricContext` are selected together with their target labels and original attribution values. The execution device stored in the context is also forwarded to `Quantus`.

The metric returns one Local Lipschitz Estimate score for each evaluated observation. Lower values indicate that the explanation varies less than the input within the sampled neighbourhood and therefore suggest greater local stability. Higher values indicate that relatively small input changes may produce large variations in the explanation.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `LocalLipschitzEstimate` class, placed in the module `xai_metrics.metrics.robustness`. It can be executed directly using this class (Section 4.4.1.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.4.1.2).

Both usage modes rely on the same `YAML` configuration file and use the LIME explanation function created through `make_lime_explain_func()`. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. For each observation, 20 neighbour inputs are generated using Gaussian noise with a mean of (0.0) and a standard deviation of (0.1). The original signs of the attribution values are preserved, and attribution normalisation is enabled.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/LocalLipschitzEstimate_examples.py`

The explanation function used by these examples is available under: `examples/specific_examples/metrics/make_explain_func.py`

4.4.1.1 Direct evaluation using the LocalLipschitzEstimate class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric. The training data are also loaded to create the LIME explanation function required to recompute the explanations of the perturbed inputs.

After constructing the context, the `LocalLipschitzEstimate` class is instantiated with the desired parameters and the explanation function returned by `make_lime_explain_func()`. Calling the `run()` method returns one local Lipschitz estimate score for each evaluated observation, and the mean of these scores can be calculated as an aggregated result.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.config import ConfigController
from xai_metrics.metrics.robustness import LocalLipschitzEstimate

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = LocalLipschitzEstimate(
    context=context,
    params={
        "nr_samples": 20,
        "abs": False,
        "normalise": True,
        "perturb_mean": 0.0,
        "perturb_std": 0.1
    },
    explain_func = make_lime_explain_func()
)
scores = metric.run()
```

4.4.1.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file.

The explanation function cannot be defined directly in the `YAML` file. Therefore, it is created in Python and passed to `run_evaluation()` through the `explain_func` dependency. The runner subsequently provides this function when instantiating metrics that require explanations to be recomputed.

The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"LocalLipschitzEstimate"` metric, although this argument would not be necessary if `LocalLipschitzEstimate` is the only metric defined in the configuration.

```python
from make_explain_func import make_lime_explain_func
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["LocalLipschitzEstimate"],
    config=config_path,
    report_output_dir=None,
    explain_func = make_lime_explain_func()
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

metrics:
  - name: LocalLipschitzEstimate
    params:
      nr_samples: 20
      abs: false
      normalise: true
      perturb_mean: 0.0
      perturb_std: 0.1
```

4.4.2 Max Sensitivity

The Max Sensitivity metric evaluates the robustness of an explanation by measuring the largest relative change produced in the explanation when small random perturbations are applied to the corresponding input [17]. For each evaluated observation, several perturbed inputs are generated, and their explanations are recomputed using an explanation function provided by the user.

For each perturbed input, the metric compares the new attribution vector with the original explanation. The relative change is calculated as the norm of the difference between both explanations divided by the norm of the original explanation. The Max Sensitivity score is the maximum relative change obtained across all sampled perturbations:

$$\max_{x' \in \mathcal{N}(x)} \frac{\|\Phi(x) - \Phi(x')\|_F}{\|\Phi(x)\|_F},$$

where (x) is the original observation, (x') is a randomly perturbed version of that observation, $(\mathcal{N}(x))$ is the set of sampled neighbours, and (Φ) represents the explanation function. The Frobenius norm is used to measure both the explanation difference and the magnitude of the original explanation.

A low Max Sensitivity score indicates that none of the sampled perturbations produces a large relative change in the explanation and therefore suggests greater robustness. Conversely, a high score indicates that at least one small perturbation produces a substantially different explanation. Since only a finite number of perturbed inputs are evaluated, the resulting value is an approximation of the maximum sensitivity within the sampled neighbourhood.

The parameters used by this metric can be summarized as follows:

- `nr_samples (int)`: number of perturbed inputs generated for each evaluated observation. Increasing this value explores a larger number of possible variations but also increases the computational cost, since a new explanation must be generated for every perturbed input. (*Default:* `200`)
- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)
- `normalise (bool)`: whether to normalise the original and perturbed attribution values before comparing them. When enabled, the default normalisation function provided by `Quantus` is used. (*Default:* `False`)
- `lower_bound (float)`: lower bound used by the uniform-noise perturbation function when generating perturbed inputs. This parameter controls the range from which the random perturbations are sampled. (*Default:* `0.2`)
- `upper_bound (float or None)`: optional upper bound used by the uniform-noise perturbation function. When its value is `None`, the default behaviour defined by `Quantus` is used. (*Default:* `None`)

In addition to these parameters, the metric requires an external explanation function, provided through the `explain_func` argument. This function is necessary because the explanations must be recomputed after perturbing each input. It must be compatible with the interface expected by `Quantus` and use the following function signature:

```
def explain_func(model, inputs, targets=None, **kwargs):
...
```

The `model` argument contains the model being evaluated, `inputs` contains the observations whose explanations must be generated, and `targets` optionally identifies the target outputs. The function must return a `NumPy array` containing the attribution vectors, with one row for each input observation. Additional arguments may be received through `**kwargs` for compatibility with the `Quantus` interface. If no explanation function is provided, the constructor raises a `ValueError`.

The implementation uses the default comparison, norm, normalisation, and perturbation functions provided by `Quantus`. Explanation changes are calculated using an element-wise difference, their magnitude is measured using the Frobenius norm, and perturbed observations are generated using uniform noise with the configured bounds.

Before computing the metric, the model is set to training mode, following the current implementation of the wrapper. The observations specified in the `MetricContext` are selected together

with their target labels and original attribution values. The explanation function and the execution device stored in the context are then forwarded to `Quantus`.

The metric returns one max sensitivity score for each evaluated observation. Lower values indicate that the explanations remain comparatively stable across all the sampled perturbations. Higher values indicate that at least one perturbed input produces a large relative change in the corresponding explanation.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `MaxSensitivity` class, placed in the module `xai_metrics.metrics.robustness`. It can be executed directly using this class (Section 4.4.2.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.4.2.2).

Both usage modes rely on the same `YAML` configuration file and use the LIME explanation function created through `make_lime_explain_func()`. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. For each observation, 20 perturbed inputs are generated using uniform noise with a lower bound of (0.2). The original signs of the attribution values are preserved, and attribution normalisation is disabled. Since no value is assigned to `upper_bound`, the default behaviour provided by `Quantus` is used.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/MaxSensitivity_examples.py`

The explanation function used by these examples is available under: `examples/specific_examples/metrics/make_explain_func.py`

4.4.2.1 Direct evaluation using the MaxSensitivity class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric. The training data are also loaded to create the LIME explanation function required to recompute the explanations of the perturbed inputs.

After constructing the context, the `MaxSensitivity` class is instantiated with the desired parameters and the explanation function returned by `make_lime_explain_func()`. Calling the `run()` method returns one max sensitivity score for each evaluated observation, and the mean of these scores can be calculated as an aggregated result.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.config import ConfigController
from xai_metrics.metrics.robustness import MaxSensitivity

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = MaxSensitivity(
    context=context,
    params={
        "nr_samples": 20,
        "abs": False,
        "normalise": False,
        "lower_bound": 0.2
    },
    explain_func = make_lime_explain_func()
```

```
)
scores = metric.run()
```

4.4.2.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file.

The explanation function cannot be defined directly in the `YAML` file. Therefore, it is created in Python and passed to `run_evaluation()` through the `explain_func` dependency. The runner subsequently provides this function when instantiating metrics that require explanations to be recomputed.

The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"MaxSensitivity"` metric, although this argument would not be necessary if `MaxSensitivity` is the only metric defined in the configuration.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["MaxSensitivity"],
    config=config_path,
    report_output_dir=None,
    explain_func = make_lime_explain_func()
)
```

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

metrics:
  - name: MaxSensitivity
    params:
      nr_samples: 20
      abs: false
      normalise: false
      lower_bound: 0.2
```

4.4.3 Relative Input Stability

The Relative Input Stability metric evaluates explanation robustness by comparing the relative change in an explanation with the relative change in its corresponding input [18]. For each evaluated observation, several perturbed inputs are generated, and their explanations are recomputed using an explanation function provided by the user.

For each perturbed input, the metric calculates the ratio between the relative variation in the explanation and the relative variation in the input. The Relative Input Stability score is the maximum ratio obtained across all the sampled perturbations:

$$\max_{x' \in \mathcal{N}(x)} \frac{\left\| \frac{\Phi(x) - \Phi(x')}{\Phi(x)} \right\|_p}{\max\left(\left\| \frac{x - x'}{x} \right\|_p, \epsilon \right)},$$

where (x) is the original observation, (x') is a perturbed observation, $(\mathcal{N}(x))$ is the set of sampled neighbours, and (Φ) represents the explanation function. The divisions in the numerator and denominator are performed element-wise. A small constant (ϵ) is used to avoid divisions by zero.

A low Relative Input Stability score indicates that the relative change in the explanation is small compared with the relative change in the input and therefore suggests greater robustness. Conversely, a high score indicates that comparatively small relative changes in the input may produce large relative changes in the explanation.

Relative Input Stability considers their relative changes. This reduces the influence of differences in the scale of the input features and attribution values. Since the metric evaluates only a finite number of randomly perturbed inputs, the resulting value is an approximation of the maximum relative instability within the sampled neighbourhood.

The parameters used by this metric can be summarized as follows:

- `nr_samples (int)`: number of perturbed inputs generated for each evaluated observation. Increasing this value provides a more extensive exploration of the local neighbourhood but also increases the computational cost, since an explanation must be generated for every perturbed input. (*Default:* `200`)
- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)
- `normalise (bool)`: whether to normalise the original and perturbed attribution values before calculating their relative changes. When enabled, the `normalise_by_average_second_moment_estimate()` function provided by `Quantus` is used. (*Default:* `False`)

In addition to these parameters, the metric requires an external explanation function, provided through the `explain_func` argument. This function is necessary because the explanations must be recomputed after perturbing each input. It must be compatible with the interface expected by `Quantus` and use the following function signature:

```
1 def explain_func(model, inputs, targets=None, **kwargs):
2 ...
```

The `model` argument contains the model being evaluated, `inputs` contains the observations whose explanations must be generated, and `targets` optionally identifies the target outputs. The function must return a `NumPy array` containing the generated attribution vectors, with one row for each input observation. Additional arguments may be received through `**kwargs` for compatibility with the `Quantus` interface. If no explanation function is provided, the constructor raises a `ValueError`.

The implementation uses the default perturbation and stability functions provided by `Quantus`. Neighbouring inputs are generated by adding uniform noise with an upper bound of (0.2). The relative differences in the inputs and explanations are measured using the corresponding vector norms. A constant of (10^{-6}) is used to prevent divisions by zero.

By default, perturbations that change the model prediction are not considered valid stability comparisons. Their individual results are assigned a `nan` value. Consequently, the final score for an observation may also be `nan` when one of the sampled perturbations changes its prediction.

Before computing the metric, the model is set to evaluation mode. The observations specified in the `MetricContext` are selected together with their target labels and original attribution values. The explanation function and the execution device stored in the context are then forwarded to `Quantus`.

The metric returns one relative input stability score for each evaluated observation. Lower values indicate that the explanations vary less than the inputs in relative terms and therefore suggest greater stability. Higher values indicate that small relative input variations may produce comparatively large changes in the explanations.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `RelativeInputStability` class, placed in the module `xai_metrics.metrics.robustness`. It can be executed directly using this class (Section 4.4.4.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.4.3.2).

Both usage modes rely on the same `YAML` configuration file and use the LIME explanation function created through `make_lime_explain_func()`. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. For each observation, 20 perturbed inputs are generated. The original signs of the attribution values are preserved, and attribution normalisation is disabled.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/RelativeInputStability_examples.py`

The explanation function used by these examples is available under: `examples/specific_examples/metrics/make_explain_func.py`

4.4.3.1 **Direct evaluation using the RelativeInputStability class**

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric. The training data are also loaded to create the LIME explanation function required to recompute the explanations of the perturbed inputs.

After constructing the context, the `RelativeInputStability` class is instantiated with the desired parameters and the explanation function returned by `make_lime_explain_func()`. Calling the `run()` method returns one relative input stability score for each evaluated observation, and the mean of these scores can be calculated as an aggregated result.

```python
from make_explain_func import make_lime_explain_func
from xai_metrics.config import ConfigController
from xai_metrics.metrics.robustness import RelativeInputStability

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = RelativeInputStability(
    context=context,
    params={
        "nr_samples": 20,
        "abs": False,
        "normalise": False
    },
    explain_func = make_lime_explain_func()
)
scores = metric.run()
```

4.4.3.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file.

The explanation function cannot be defined directly in the `YAML` file. Therefore, it is created in Python and passed to `run_evaluation()` through the `explain_func` dependency. The runner subsequently provides this function when instantiating metrics that require explanations to be recomputed.

The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"RelativeInputStability"` metric, although this argument would not be necessary if `RelativeInputStability` is the only metric defined in the configuration.

```python
from make_explain_func import make_lime_explain_func
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["RelativeInputStability"],
    config=config_path,
    report_output_dir=None,
    explain_func = make_lime_explain_func()
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"
```

```yaml
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

metrics:
  - name: RelativeInputStability
    params:
      nr_samples: 20
      abs: false
      normalise: false
```

4.4.4 **Relative Output Stability**

The Relative Output Stability metric evaluates explanation robustness by comparing the relative change in an explanation with the corresponding change in the output of the predictive model [18]. For each evaluated observation, several perturbed inputs are generated, and their explanations and model outputs are recomputed.

For each perturbed input, the metric calculates the ratio between the relative change in the explanation and the distance between the original and perturbed model-output vectors. The Relative Output Stability score is the maximum ratio obtained across all sampled perturbations that preserve the original prediction:

$$\max_{x' \in \mathcal{N}(x)\ \hat{y}x' = \hat{y}x} \frac{\left\| \frac{\Phi(x) - \Phi(x')}{\Phi(x)} \right\|_p}{\max\left(\left\| h(x) - h(x') \right\|_p, \epsilon_{\min} \right)},$$

where (x) is the original observation, (x') is a perturbed neighbour, $(\mathcal{N}(x))$ is the set of sampled neighbours, (Φ) represents the explanation function, and $(h(x))$ represents the model-output vector for (x). The division in the numerator is performed element-wise, and $(\epsilon_{\min})$ is a small constant used to avoid division by zero.

A low Relative Output Stability score indicates that the explanation changes only slightly in relation to the corresponding variation in the model output and therefore suggests greater robustness. Conversely, a high score indicates that a comparatively small change in the model output produces a large relative change in the explanation.

Relative Output Stability uses the behaviour of the predictive model as its reference. Consequently, changes in an explanation are not necessarily considered undesirable when they are accompanied by substantial changes in the model output. Since only a finite number of perturbed inputs are evaluated, the resulting value is an approximation of the maximum relative instability within the sampled neighbourhood.

The parameters used by this metric can be summarized as follows:

- `nr_samples (int)`: number of perturbed inputs generated for each evaluated observation. Increasing this value provides a more extensive exploration of the local neighbourhood but also increases the computational cost, since the model output and explanation must be recomputed for every perturbation. (*Default:* `200`)

- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)
- `normalise (bool)`: whether to normalise the original and perturbed attribution values before calculating their relative changes. When enabled, the `normalise_by_average_second_moment_estimate` function provided by `Quantus` is used. (*Default:* `False`)

In addition to these parameters, the metric requires an external explanation function, provided through the `explain_func` argument. This function is necessary because the explanations must be recomputed after perturbing each input. It must be compatible with the interface expected by `Quantus` and use the following function signature:

```
def explain_func(model, inputs, targets=None, **kwargs):
...
```

The implementation uses the default perturbation, normalisation, and stability functions provided by `Quantus`. Neighbouring inputs are generated by adding uniform noise with an upper bound of (0.2). The denominator measures the distance between the original and perturbed model-output vectors, and a constant of (10^{-6}) is used to avoid division by zero.

The metric is intended to compare explanations only when the original and perturbed observations receive the same predicted class. By default, a perturbation that changes the predicted class produces a `nan` result. Therefore, the score returned for an observation may be `nan` when its prediction is not preserved across the sampled perturbations.

Before computing the metric, the model is set to evaluation mode. The observations specified in the `MetricContext` are selected together with their target labels and original attribution values. The explanation function and the execution device stored in the context are then forwarded to `Quantus`.

The metric returns one relative output stability score for each evaluated observation. Lower values indicate that the explanations remain stable relative to changes in the model output. Higher values indicate that small variations in the model output may be accompanied by comparatively large changes in the explanation.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `RelativeOutputStability` class, placed in the module `xai_metrics.metrics.robustness`. It can be executed directly using this class (Section **??**) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.4.4.2).

Both usage modes rely on the same `YAML` configuration file and use the LIME explanation function created through `make_lime_explain_func()`. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. For each observation, 20 perturbed inputs are generated. The original signs of the attribution values are preserved, and attribution normalisation is disabled.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/RelativeOutputStability_examples.py`

The explanation function used by these examples is available under: `examples/specific_examples/metrics/make_explain_func.py`

4.4.4.1 Direct evaluation using the RelativeOutputStability class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric. The training data are also loaded to create the LIME explanation function required to recompute the explanations of the perturbed inputs.

After constructing the context, the `RelativeOutputStability` class is instantiated with the desired parameters and the explanation function returned by `make_lime_explain_func()`. Calling the `run()` method returns one relative output stability score for each evaluated observation, and the mean of these scores can be calculated as an aggregated result.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.config import ConfigController
from xai_metrics.metrics.robustness import RelativeOutputStability

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = RelativeOutputStability(
    context=context,
    params={
        "nr_samples": 20,
        "abs": False,
        "normalise": False
    },
    explain_func = make_lime_explain_func()
)
scores = metric.run()
```

4.4.4.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file.

The explanation function cannot be defined directly in the `YAML` file. Therefore, it is created in Python and passed to `run_evaluation()` through the `explain_func` dependency. The runner subsequently provides this function when instantiating metrics that require explanations to be recomputed.

The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"RelativeOutputStability"` metric, although this argument would not be necessary if `RelativeOutputStability` is the only metric defined in the configuration.

```
from make_explain_func import make_lime_explain_func
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["RelativeOutputStability"],
    config=config_path,
    report_output_dir=None,
    explain_func = make_lime_explain_func()
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

metrics:
  - name: RelativeOutputStability
    params:
      nr_samples: 20
      abs: false
      normalise: false
```

4.5 Fidelity

This category includes metrics that evaluate the extent to which an explanation accurately represents the behaviour of the predictive model [14]. Fidelity considers whether the information provided by an explanation is consistent with the model's actual decision-making process and whether it is sufficient to characterize the factors that influence its predictions.

For attribution-based explanations, fidelity metrics analyse whether the relevance assigned to the input features corresponds to their actual role in the model. A highly faithful explanation should identify the information used by the model without introducing unsupported relationships or omitting relevant aspects of the prediction process.

Fidelity is particularly important because an explanation may be understandable and visually coherent without accurately reflecting the model being explained. Therefore, these metrics help identify explanation methods that generate plausible interpretations but fail to represent the actual behaviour of the predictive model.

Following the adopted taxonomy, fidelity is divided into two complementary subcategories: completeness and soundness. Completeness evaluates whether the explanation captures sufficient information about the model decision, whereas soundness evaluates whether the information included in the explanation is correct and supported by the model behaviour.

Fidelity scores may depend on the type of explanation, the model architecture, the perturbation or comparison strategy, and the definition of relevant information used by each metric. In addition, some metrics evaluate fidelity locally for individual observations, while others analyse more general properties of the explanation method. Consequently, the results should be interpreted considering the specific aspect of model fidelity evaluated by each metric and the characteristics of the experimental configuration.

4.5.1 **Completeness**

Completeness metrics evaluate whether an explanation contains sufficient information to represent the relevant behaviour of the predictive model or to cover the elements involved in its decision-making process [14]. An explanation is considered complete when the information it provides is sufficient to account for the model prediction, even if the explanation does not necessarily reproduce every internal operation of the model.

For attribution-based explanations, completeness commonly refers to whether the selected features and their assigned relevance values capture all the information required to explain the prediction. An incomplete explanation may correctly identify some influential features while omitting other factors that also contribute substantially to the model output.

These metrics are useful for detecting explanations that provide only a partial view of the model decision. However, completeness does not by itself guarantee that all the information included in an explanation is correct. An explanation may contain enough information to reproduce or characterize a prediction while still assigning relevance to features that are not genuinely supported by the model. Completeness should therefore be analysed together with soundness.

The interpretation of completeness depends on how sufficient information is defined by each metric. Some approaches examine whether the explanation covers the model's relevant decision elements, whereas others evaluate whether the explanation can reproduce the model output or distinguish between its possible predictions.

4.5.1.1 **Completeness**

The Completeness metric evaluates whether the attribution values account for the difference between the model output for an original observation and the output obtained for a baseline input [14]. This property is also known as *Summation to Delta* or *Conservation*.

For each evaluated observation, a reference input is constructed by replacing all its features with values determined by a selected baseline strategy. The metric then compares the sum of the feature attributions with the difference between the model outputs for the original and baseline inputs:

$$\sum_{i=1}^{d} \Phi_i(x) \approx f_y(x) - f_y(x_{\text{base}}),$$

where (x) is the original observation, (x_{base}) is the baseline input, $(\Phi_i(x))$ is the attribution assigned to feature (i), and $(f_y(x))$ is the model output associated with the evaluated target.

The metric returns a boolean value for each observation. A value of `True` indicates that the sum of the attribution values satisfies the completeness condition, whereas `False` indicates that the attribution sum does not match the difference between the original and baseline model outputs. When the results are aggregated, their mean represents the proportion of observations that satisfy this property. Therefore, higher values are better.

The parameters used by this metric can be summarized as follows:

- `abs (bool)`: whether to use the absolute values of the attributions before computing the metric. If all attribution values are negative and this parameter is enabled, their absolute values are used. Otherwise, the metric is skipped. (*Default:* `False`)

- `normalise (bool)`: whether to normalise the attribution values before comparing their sum with the difference between the model outputs. When enabled, the default normalisation function provided by `Quantus` is used. (*Default:* `True`)
- `perturb_baseline (str)`: strategy used to construct the reference input by replacing all its feature values. Common supported values include `"black"`, `"white"`, `"mean"`, `"random"`, and `"uniform"`. (*Default:* `"black"`)

The implementation uses the default perturbation and output-transformation functions provided by `Quantus`. The baseline input is generated by replacing all features according to the configured strategy, while the identity function is used to transform the model outputs. Therefore, the metric uses the outputs returned by the model wrapper without applying a softmax transformation.

Before computing the metric, the model is set to evaluation mode. The observations specified in the `MetricContext` are selected together with their target labels and attribution values and are passed to `Quantus`.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric is implemented through the `Completeness` class, which can be found in the module `xai_metrics.metrics.fidelity.completeness`. It can be executed directly using this class (Section 4.5.1.1.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.5.1.1.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. The original signs of the attribution values are preserved, attribution normalisation is enabled, and the default `"black"` baseline is used.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/Completeness_examples.py`

4.5.1.1.1 Direct evaluation using the Completeness class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `Completeness` class is instantiated directly using the desired parameters. Calling the `run()` method returns one completeness score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.fidelity.completeness import Completeness

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = Completeness(
    context=context,
    params={
        "abs": False,
        "normalise": True
    }
)
scores = metric.run()
```

4.5.1.1.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"Completeness"` metric, although this argument would not be necessary if `Completeness` is the only metric defined in the configuration.

```python
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["Completeness"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: Completeness
    params:
      abs: false
      normalise: true
```

4.5.2 Soundness

Soundness metrics evaluate whether the information contained in an explanation is correct and supported by the actual behaviour of the predictive model. A sound explanation should not assign relevance to features, concepts, or relationships that do not influence the model prediction.

For attribution-based explanations, soundness analyses whether the features identified as relevant genuinely affect the model output and whether the explanation reacts consistently to changes in the model or the input. This makes it possible to detect explanations that appear reasonable but are insensitive to the behaviour of the model being explained.

Soundness differs from completeness in the type of error it seeks to identify. Completeness focuses on whether relevant information has been omitted, whereas soundness focuses on whether the information included in the explanation is valid. An explanation may therefore be sound but incomplete if every reported feature is genuinely relevant but some important features are missing. Conversely, it may be complete but unsound if it contains sufficient relevant information together with additional unsupported elements.

Soundness metrics may use different evaluation strategies. Some perturb the input features and examine changes in the model output, while others modify model parameters, compare explanations produced by functionally equivalent models, or evaluate agreement between attribution values and independently estimated feature relevance. As a result, the scores may depend on the perturbation baseline, the model-output representation, the similarity measure, and the characteristics of the data.

4.5.2.1 **Non-Sensitivity**

The Non-Sensitivity metric evaluates whether features with negligible attribution values correspond to features on which the selected model output does not functionally depend [13]. For each evaluated observation, the metric identifies the features whose attribution values are below a threshold and compares them with the features whose perturbation produces only a negligible change in the model output.

When one feature is perturbed at each step, the two sets considered by the metric can be expressed as:

$$\mathcal{I}_{\Phi}(x) = \{i \,|\, |\Phi_i(x)| < \varepsilon\}\,,$$

$$\mathcal{I}_{f}(x) = \left\{i \,\middle|\, \left|f_y(x) - f_y\left(x^{(i)}\right)\right| < \varepsilon\right\}.$$

where (x) is the original observation, $(\Phi_i(x))$ is the attribution assigned to feature (i), $(x^{(i)})$ is the observation obtained after replacing feature (i) with a baseline value, and (f_y) is the model output associated with the evaluated target.

The Non-Sensitivity score corresponds to the number of disagreements between these sets:

$$|\mathcal{I}\Phi(x) \triangle \mathcal{I}f(x)|\,,$$

where $(\triangle)$ denotes the symmetric difference between two sets. A disagreement occurs when a feature receives a negligible attribution even though perturbing it changes the model output, or when a feature receives a non-negligible attribution despite having no significant effect on the selected output.

A low Non-Sensitivity score indicates stronger agreement between the attribution values and the functional behaviour of the model. A score of (0) indicates that every feature classified as irrelevant by the explanation is also identified as irrelevant through perturbation, and vice versa. Higher values indicate a larger number of disagreements and therefore weaker compliance with the non-sensitivity property.

The parameters used by this metric can be summarized as follows:

- `eps (float)`: threshold used both to identify negligible attribution values and to determine whether the change in the model output after perturbation is insignificant. Attribution values below this threshold are considered negligible, while output differences smaller than the same threshold indicate that the model is insensitive to the perturbed feature. (*Default:* `1e-5`)
- `features_in_step (int)`: number of features replaced simultaneously at each perturbation step. A value of 1 evaluates each feature individually, whereas larger values perturb groups of features. (*Default:* `1`)

- `abs (bool)`: whether to use the absolute attribution values before identifying negligible features. When enabled, the direction of an attribution is ignored and only its magnitude is considered. If all attribution values are negative and this parameter is disabled, the metric is skipped. (*Default:* `True`)

- `normalise (bool)`: whether to normalise the attribution values before applying the `eps` threshold. When enabled, the default `normalise_by_max` function provided by `Quantus` is used. (*Default:* `True`)

- `perturb_baseline (str)`: baseline used to replace the selected features during the perturbation process. Common supported values include `"black"`, `"white"`, `"mean"`, `"random"`, and `"uniform"`. (*Default:* `"black"`)

The implementation uses the default perturbation and aggregation functions provided by `Quantus`. At each step, the selected features are replaced with values determined by the configured baseline. The resulting model output is compared with the output obtained for the original observation, and differences smaller than `eps` are treated as negligible.

Before computing the metric, the model is set to evaluation mode. The observations specified in the `MetricContext` are selected together with their target labels and attribution values and are then passed to `quantus.NonSensitivity`.

When all attribution values are negative, their treatment depends on the `abs` parameter. If `abs=True`, their absolute values are used and the evaluation continues. If `abs=False`, the metric raises a `MetricSkipped` exception and is not evaluated for that attribution configuration.

The metric returns one Non-Sensitivity score for each evaluated observation. Lower values indicate better agreement between features with negligible attributions and features that do not significantly affect the model output. The mean of the returned scores represents the average number of disagreements per observation.

Since the score is a count rather than a normalised proportion, its possible magnitude depends on the number of input features and on the value of `features_in_step`. Therefore, direct comparisons between datasets with different dimensionalities should be made with caution.

The metric is implemented through the `NonSensitivity` class, which can be found in the module `xai_metrics.metrics.fidelity.soudness`. It can be executed directly using this class (Section 4.5.2.1.1) or as part of the automated evaluation pipeline provided by `run_evaluation()` (Section 4.5.2.1.2).

Both usage modes rely on the same `YAML` configuration file. The experiment evaluates explanations generated by LIME for an Isolation Forest model on the MetroPT3 dataset. Each feature is perturbed individually using the `"black"` baseline. Absolute attribution values and attribution normalisation are enabled, and a threshold of (10^{-5}) is used to identify negligible attributions and model-output changes.

Examples of both usage modes are available in the library under: `examples/specific_examples/metrics/NonSensitivity_examples.py`

4.5.2.1.1 Direct evaluation using the NonSensitivity class

In the direct evaluation mode, the experimental context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, test data, labels, attribution values, selected observations, and execution device required to calculate the metric.

After constructing the context, the `NonSensitivity` class is instantiated directly using the desired parameters. Calling the `run()` method returns one non-sensitivity score for each evaluated observation, and the mean of these scores is calculated as an aggregated result.

```
from xai_metrics.config import ConfigController
from xai_metrics.metrics.fidelity.soundness import NonSensitivity

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_metric_context()
metric = NonSensitivity(
    context=context,
    params={
        "eps": 1e-5,
        "features_in_step": 1,
        "abs": True,
        "normalise": True,
        "perturb_baseline": "black"
    }
)
scores = metric.run()
```

4.5.2.1.2 Automated evaluation using run_evaluation()

The automated evaluation mode uses the `run_evaluation()` function. The experimental context, the metrics to be executed, and their parameters are obtained from the `YAML` configuration file. The optional `selected_metrics` argument can be used to restrict the evaluation to a subset of the configured metrics. In this example, it selects only the registered `"NonSensitivity"` metric, although this argument would not be necessary if `NonSensitivity` is the only metric defined in the configuration.

```
from xai_metrics.runner import run_evaluation

config_path = "examples/specific_examples/config.yaml"
results = run_evaluation(
    selected_metrics=["NonSensitivity"],
    config=config_path,
    report_output_dir=None
)
```

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_test_path: "examples/datasets/MetroPT3/X_test.csv"
  y_test_path: "examples/datasets/MetroPT3/y_test.csv"
```

```
  attributions_path: "examples/attributions/MetroPT3/IForest/LIME/
      MetroPT3_IForest_lime_attributions.csv"

metrics:
  - name: NonSensitivity
    params:
      eps: 1e-5
      features_in_step: 1
      abs: true
      normalise: true
      perturb_baseline: black
```

CHAPTER

5

Run available XAI methods in the Library

In this chapter, the explanation methods implemented in the **XAIMetrics** library are presented. The current version of the library includes four local, post-hoc, attribution-based methods for tabular data: LIME, SHAP, BreakDown, and MAPLE. Each method receives a predictive model and a set of observations and returns an attribution matrix in which every row corresponds to an explained observation and every column corresponds to an input feature.

For each explanation method, this chapter first describes its main objective and internal operation. The parameters used by the corresponding wrapper are then summarized, together with the most relevant implementation considerations. Finally, two usage modes are presented. The first executes the explainer directly through its Python class, whereas the second includes it in an automated explanation experiment configured through a `YAML` file and executed using `run_explanation()`.

All explanation methods inherit from the `BaseExplainer` class and receive an `ExplainerContext`. This context groups the predictive model, the background data, the observations to explain, their associated target values, and the execution device. The explanation methods implement a common `explain()` method and are registered automatically, allowing them to be selected by name in the experimental configuration.

In addition, the `as_explain_func()` method converts an explainer into a callable compatible with evaluation metrics that must recompute explanations for modified inputs. The resulting callable follows the interface expected by `Quantus`:

```
1 def explain_func(model, inputs, targets=None, **kwargs):
2     ...
```

The `model` argument contains the model being evaluated, `inputs` contains the observations whose explanations must be generated, and `targets` optionally identifies the model outputs to explain. The function returns a `NumPy` array with one attribution vector per input observation.

5.1 LIME

LIME (*Local Interpretable Model-agnostic Explanations*) generates a local explanation for an individual observation by approximating the behaviour of the original predictive model in a neighbourhood around that observation [5]. Instead of attempting to explain the complete model globally, LIME generates perturbed samples close to the observation and fits a simpler interpretable model using the predictions produced for those samples.

The local surrogate model is obtained by minimizing an objective that combines its approximation error around the observation with a complexity penalty:

$$\xi(x)\arg,\min_{g\in\mathcal{G}};\mathcal{L}\left(f,g,\pi_x\right)+\Omega(g),$$

where (x) is the observation being explained, (f) is the original predictive model, (g) is an interpretable local surrogate model belonging to the family $(\mathcal{G})$, (π_x) assigns greater importance to perturbed samples that are closer to (x), $(\mathcal{L})$ measures how well (g) approximates (f) in the local neighbourhood, and $(\Omega(g))$ penalizes the complexity of the surrogate model.

For tabular data, LIME generates a set of perturbed observations around the input and obtains their predictions from the original model. Each perturbed sample receives a weight according to its distance from the original observation. A weighted local model is then fitted, and its coefficients are used as feature-attribution values.

A positive attribution indicates that the corresponding feature contributes towards the selected target output in the local surrogate model, whereas a negative attribution indicates a contribution in the opposite direction. The magnitude of the attribution represents the local importance assigned to the feature. Since LIME approximates the model only within a sampled neighbourhood, its results may vary depending on the perturbation process, the random seed, the kernel, and the number of generated samples.

The implementation provided by `XAIMetrics` uses the `LimeTabularExplainer` class from the `lime` package. The background data stored in the `ExplainerContext` are used to initialize this object and characterize the feature distributions from which the perturbed observations are generated.

The columns of `context.X_background` determine the order of the returned attribution values. For every explained observation, the implementation creates a vector with one position per input feature. Features omitted from the local explanation, for example because `num_features` is smaller than the total number of features, receive an attribution value of (0).

The parameters supported by the explainer can be summarized as follows:

- `mode (str)`: type of predictive task explained by LIME. The usual supported values are `"classification"` and `"regression"`. In classification mode, the explainer requires a probability-prediction function. In regression mode, it uses the standard prediction function. (*Default:* `"classification"`)
- `categorical_features (list or None)`: indices of the features that should be treated as categorical. These indices refer to the column order of `X_background`. (*Default:* `None`)
- `categorical_names (dict or None)`: mapping from categorical feature indices to the names of their possible categories. (*Default:* `None`)

- `kernel_width (float or None)`: width of the proximity kernel used to weight the perturbed observations. Smaller values give more importance to samples very close to the explained observation, whereas larger values define a wider local neighbourhood. If no value is provided, LIME determines its default width from the number of features. (*Default:* `None`)

- `kernel (callable or None)`: optional custom kernel function used to transform distances into proximity weights. Since Python callables cannot be defined directly in a standard `YAML` file, this option is mainly intended for direct class instantiation. (*Default:* `None`)

- `verbose (bool)`: whether the underlying LIME implementation should print additional information during the construction of the local explanation. (*Default:* `False`)

- `class_names (list or None)`: names assigned to the model classes in classification mode. This parameter affects the representation of the explanation but not the order of the returned attribution matrix. (*Default:* `None`)

- `feature_selection (str)`: strategy used by LIME to select the features included in the local surrogate model. Supported strategies depend on the underlying LIME implementation and include options such as `"auto"`, `"forward_selection"`, `"highest_weights"`, `"lasso_path"`, and `"none"`. (*Default:* `"auto"`)

- `discretize_continuous (bool)`: whether continuous variables should be discretized before generating the interpretable representation used by LIME. (*Default:* `True`)

- `discretizer (str)`: strategy used to discretize continuous features. Common options supported by LIME include `"quartile"`, `"decile"`, and `"entropy"`. This parameter is used only when continuous-feature discretization is enabled. (*Default:* `"quartile"`)

- `sample_around_instance (bool)`: whether continuous perturbations should be centred around the observation being explained instead of around the mean of the background data. (*Default:* `False`)

- `random_state (int)`: random seed used during the perturbation and local-model construction processes. Fixing this value improves the reproducibility of the generated explanations. (*Default:* `42`)

- `labels (tuple, list or None)`: target labels for which explanations should be generated. When this parameter is not provided and `top_labels` is also disabled, the implementation uses the target associated with each observation. If no target values are supplied, class 1 is explained by default. (*Default:* `None`)

- `top_labels (int or None)`: number of classes with the highest predicted values for which LIME should generate explanations. When this option is enabled, the `XAIMetrics` wrapper uses the first class returned in `explanation.top_labels` to construct the attribution vector. (*Default:* `None`)

- `num_features (int)`: maximum number of features included in each local explanation. Features not selected by LIME are represented with zero values in the returned attribution matrix. (*Default:* number of input features)

- `num_samples (int)`: number of perturbed observations generated to fit each local surrogate model. Larger values may provide a more stable approximation of the local behaviour but increase the computational cost. A separate perturbation dataset and local model are generated for every explained observation. (*Default:* `500`)

- `distance_metric (str)`: distance metric used to calculate the proximity between the original observation and its perturbed samples. (*Default:* `"euclidean"`)

- `model_regressor (Any or None)`: optional regressor used to fit the local surrogate model. The regressor must support sample weights and expose its fitted coefficients. Since this object cannot normally be represented directly in `YAML`, it is mainly intended for direct class usage. (*Default:* `None`)

The explainer accepts input observations represented as a `pandas.DataFrame`, a PyTorch `Tensor`, a `NumPy` array, or another array-like object. Data frames are reordered according to the columns of `X_background`, while tensors are detached and transferred to the CPU before being converted into `NumPy` arrays. A one-dimensional input is reshaped automatically into a batch containing one observation.

The prediction function used by LIME depends on the configured mode. In classification mode, the implementation searches for a `predict_proba()` method. In any other mode, it searches for `predict()`. The required method is first searched directly on the supplied model and then on its internal `model` attribute. If the required method is unavailable in both locations, the explainer raises an `AttributeError`.

For each input observation, the selected label is determined according to the configured parameters. If `top_labels` is provided, the first class returned by LIME is used. Otherwise, the first value specified in `labels` is selected. When neither parameter is configured, the target passed to `explain()` is used for each observation. If no target is available, label (1) is selected by default.

The local feature weights returned by `LimeTabularExplainer` are converted into a fixed-size attribution vector. Each weight is placed in the position corresponding to its original feature index, ensuring that every returned row follows the column order of the background dataset.

The explainer returns a two-dimensional `NumPy` array with one row for each explained observation and one column for each input feature:

$$A \in \mathbb{R}^{n \times d},$$

where (n) is the number of observations in the explained batch and (d) is the number of features in `X_background`.

The explainer is implemented through the `LIMEExplainer` class, located in the module `xai_metrics.explainers.lime`. It is registered under the name `"LIME"` and can be executed directly using this class (Section 5.1.1) or as part of the automated explanation pipeline provided by `run_explanation()` (Section 5.1.2).

Both usage modes rely on the same `YAML` configuration file. The example generates LIME explanations for an Isolation Forest model on the MetroPT3 dataset. The explainer operates in classification mode, generates 5000 perturbed samples for each observation, explains class (1), calculates distances using the Euclidean metric, and uses a fixed random seed of (42).

Examples of both usage modes are available in the library under: `examples/specific_examples/explainers/LIME_examples.py`

5.1.1 Direct explanation using the LIMEExplainer class

In the direct usage mode, the explanation context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, background data, optional background labels, observations to explain, target values, and execution device required by the explanation process.

After constructing the context, the `LIMEExplainer` class is instantiated directly using the desired parameters. Calling the `explain()` method with the model, input batch, and target values returns the complete attribution matrix.

```
from xai_metrics.config import ConfigController
from xai_metrics.explainers.lime import LIMEExplainer

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_explainers_context()
explainer = LIMEExplainer(
    context=context,
    params={
        "mode": "classification",
        "random_state": 42,
        "num_samples": 5000,
        "labels": [1],
        "distance_metric": "euclidean",
    },
)

attributions = explainer.explain(
    model=context.model,
    inputs=context.X_batch,
    targets=context.y_batch,
)
```

The resulting array contains one row for every observation in `context.X_batch` and one column for every feature in `context.X_background`. The mean absolute attribution calculated in the example provides a compact summary of the magnitude of the generated values, but it should not be interpreted as a measure of explanation quality.

5.1.2 Automated explanation using run_explanation()

The automated usage mode uses the `run_explanation()` function. The explanation context, the explainers to be executed, and their parameters are obtained from the `YAML` configuration file.

The optional `selected_explainers` argument can be used to restrict the execution to a subset of the configured explanation methods. In this example, it selects only the registered `"LIME"` explainer, although this argument would not be necessary if LIME were the only explainer defined in the configuration.

```
from xai_metrics.runner import run_explanation

config_path = "examples/specific_examples/config.yaml"
results = run_explanation(
    selected_explainers=["LIME"],
    config=config_path,
    attribution_output_dir=None,
)
```

The returned structure contains the experimental metadata and the paths of any attribution files generated during the execution. When no output directory is provided, the attributions remain available in the returned result without being explicitly exported through the argument shown in the example.

The corresponding experimental configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

explainers:
  - name: LIME
    params:
      mode: classification
      random_state: 42
      num_samples: 5000
      labels: [1]
      distance_metric: euclidean
```

The paths to `X_background` and `y_background` provide the reference data used to initialize the LIME tabular explainer. The paths to `X_batch` and `y_batch` identify the observations and targets for which the explanations are generated. The test-data and attribution paths belong to the shared experimental context and are primarily used by the metric-evaluation workflow.

5.2 SHAP

SHAP (*SHapley Additive exPlanations*) generates feature attributions by applying concepts from cooperative game theory to the explanation of predictive models [4]. Each input feature is interpreted as a player in a cooperative game, while the model prediction represents the value obtained by a coalition of features. The attribution assigned to each feature measures its average marginal contribution across the possible coalitions in which it may participate.

For a predictive model (f), the SHAP value associated with feature (i) can be expressed as:

$$\phi_i(f,x) \sum_{S \subseteq F \setminus i} \frac{|S|!\,(|F|-|S|-1)!}{|F|!} \left[f_{S \cup i}(x) f_S(x) \right],$$

where (x) is the observation being explained, (F) is the complete set of input features, (S) is a subset of features that does not contain feature (i), $(f_S(x))$ is the model output obtained when only the information in (S) is considered, and $(f_{S \cup i}(x))$ is the output obtained after adding feature (i) to that subset.

The factorial term weights the marginal contribution of feature (i) according to the size of the coalition. As a result, the SHAP value represents the average change in the model output produced by introducing that feature across all possible feature orderings.

SHAP explanations follow an additive representation:

$$f(x)\phi_0 + \sum_{i=1}^{d} \phi_i(x),$$

where (ϕ_0) is the expected model output over the background distribution, (d) is the number of input features, and ($\phi_i(x)$) is the contribution assigned to feature (i) for observation (x).

A positive SHAP value indicates that the corresponding feature increases the selected model output relative to the reference value, whereas a negative value indicates that it decreases that output. The magnitude of the value represents the strength of the contribution. However, the precise interpretation depends on the output explained by SHAP, such as a class probability, a regression prediction, or another model score.

The exact computation of Shapley values requires evaluating all possible feature subsets and is generally computationally expensive. Therefore, SHAP provides several exact and approximate algorithms adapted to different model types and computational constraints. The specific algorithm used by the library can be selected explicitly or determined automatically from the prediction function, background data, and model characteristics.

The implementation provided by `XAIMetrics` uses the generic `shap.Explainer` interface. The background data stored in the `ExplainerContext` are used to initialize the SHAP explainer and define the reference distribution against which feature contributions are calculated.

The columns of `context.X_background` determine the names and order of the features in the returned attribution matrix. The background data can optionally be subsampled before initializing SHAP, reducing the computational and memory requirements associated with large reference datasets.

The parameters supported by the explainer can be summarized as follows:

- `mode (str)`: type of predictive task explained by SHAP. If its value is `"classification"`, the wrapper uses the model's `predict_proba()` method. For any other value, it uses `predict()`. (*Default:* `"classification"`)
- `max_background_samples (int or None)`: maximum number of observations retained from the background dataset. When this parameter is provided, the background data are subsampled using `shap.sample()`. Reducing the background size can lower the computational cost, although it may also produce a less representative reference distribution. (*Default:* `None`)
- `random_state (int)`: random seed used when subsampling the background data and when initializing `shap.Explainer`. Fixing this value improves the reproducibility of algorithms that involve random sampling. (*Default:* `42`)
- `algorithm (str)`: algorithm passed to `shap.Explainer`. The value `"auto"` allows SHAP to select an appropriate algorithm automatically. Other supported values depend on the installed SHAP version and may include strategies such as `"permutation"`, `"partition"`, `"tree"`, or `"linear"`. (*Default:* `"auto"`)
- `output_names (list or None)`: optional names assigned to the model outputs. This information is passed to `shap.Explainer` and can be useful when the model produces multiple outputs. (*Default:* `None`)
- `output_index (int or None)`: output selected when SHAP returns a separate attribution value for each model output. When this parameter is provided, it takes precedence over the targets supplied to `explain()` and over the target values stored in the context. (*Default:* `None`)

- `default_output (int)`: output index selected when SHAP returns multi-output values and neither `output_index`, the targets passed to `explain()`, nor `context.y_batch` are available. (*Default:* `1`)
- `max_evals (int or None)`: maximum number of model evaluations allowed during the SHAP computation. When no value is provided, the string `"auto"` is passed to SHAP. The interpretation and minimum valid value depend on the algorithm selected by `shap.Explainer`. (*Default:* `None`)
- `batch_size (int or None)`: batch size used by SHAP when evaluating the model. When this parameter is not specified, the value `"auto"` is used. Larger batches may improve execution efficiency but require more memory. (*Default:* `None`)
- `abs (bool)`: whether to convert the generated SHAP values into their absolute values before returning them. When enabled, the direction of each feature contribution is discarded and only its magnitude is preserved. (*Default:* `False`)

The explainer accepts input observations represented as a `pandas.DataFrame`, a PyTorch `Tensor`, a `NumPy` array, or another array-like object. Data frames are reordered according to the columns of `X_background`. PyTorch tensors are detached, transferred to the CPU, and converted into `NumPy` arrays. One-dimensional inputs are reshaped automatically into a batch containing one observation.

The prediction function used by SHAP depends on the configured mode. In classification mode, the implementation searches for a `predict_proba()` method. In any other mode, it searches for `predict()`. The required method is first searched directly on the supplied model and then on its internal `model` attribute. If it is unavailable in both locations, the explainer raises an `AttributeError`.

The underlying `shap.Explainer` is created lazily the first time a particular model is explained. The implementation caches one explainer for each model object using its identity. Subsequent calls with the same model reuse the cached object, avoiding the repeated initialization of the SHAP explainer and its background data.

After the explanation is computed, the wrapper extracts the array stored in `explanation.values`. If this array has two dimensions, it is returned directly because it already contains one attribution vector per observation:

$$A \in \mathbb{R}^{n \times d},$$

where (n) is the number of explained observations and (d) is the number of input features.

For models with multiple outputs, SHAP may return a three-dimensional array:

$$A \in \mathbb{R}^{n \times d \times c},$$

where (c) is the number of model outputs. In this case, the wrapper reduces the result to a two-dimensional attribution matrix by selecting one output for each observation.

The selected output is determined according to the following priority:

1. The value explicitly configured through `output_index`.
2. The target values passed to the `explain()` method.

3. The target values stored in `context.y_batch`.
4. The value configured through `default_output`.

When target values are used, each observation may select a different output index. The number of targets must match the number of explained observations. Otherwise, the wrapper raises a `ValueError`.

If the values returned by SHAP do not have two or three dimensions, the implementation raises a `ValueError`, since their structure cannot be converted into the fixed attribution matrix expected by the library.

When `abs=True`, the absolute-value operation is applied after the output dimension has been selected. Therefore, the returned matrix preserves the feature structure but no longer indicates whether each feature increases or decreases the selected output.

The explainer returns a two-dimensional `NumPy` array with one row for each explained observation and one column for each feature in `X_background`. This uniform representation makes the resulting attribution matrix compatible with the evaluation metrics and reporting components provided by `XAIMetrics`.

The explainer is implemented through the `SHAPExplainer` class, located in the module `xai_metrics.explainers.shap`. It is registered under the name `"SHAP"` and can be executed directly using this class (Section 5.2.1) or as part of the automated explanation pipeline provided by `run_explanation()` (Section 5.2.2).

Both usage modes rely on the same `YAML` configuration file. The example generates SHAP explanations for an Isolation Forest model on the MetroPT3 dataset. The explainer operates in classification mode, allows SHAP to select the algorithm automatically, uses a maximum of 100 background observations, explains output (1), and uses a fixed random seed of (42).

Examples of both usage modes are available in the library under: `examples/specific_examples/explainers/SHAP_examples.py`

5.2.1 Direct explanation using the SHAPExplainer class

In the direct usage mode, the explanation context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, background data, optional background labels, observations to explain, target values, and execution device required by the explanation process.

After constructing the context, the `SHAPExplainer` class is instantiated directly using the desired parameters. Calling the `explain()` method with the model, input batch, and target values returns the complete attribution matrix.

```
from xai_metrics.config import ConfigController
from xai_metrics.explainers.shap import SHAPExplainer

config_path = PROJECT_ROOT / "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_explainers_context()
explainer = SHAPExplainer(
    context=context,
    params={
        "mode": "classification",
        "algorithm": "auto",
        "max_background_samples": 100,
        "output_index": 1,
```

```
        "random_state": 42,
    },
)

attributions = explainer.explain(
    model=context.model,
    inputs=context.X_batch,
    targets=context.y_batch,
)
```

The resulting array contains one row for every observation in `context.X_batch` and one column for every feature in `context.X_background`. Since `output_index` is explicitly set to (1), the targets passed to `explain()` are not used to select the explained output in this example.

The mean absolute attribution calculated in the complete example summarizes the average magnitude of the generated SHAP values. However, it should not be interpreted as a direct measure of explanation quality, since it depends on the model-output scale, the background distribution, and the number and scale of the input features.

5.2.2 Automated explanation using run_explanation()

The automated usage mode uses the `run_explanation()` function. The explanation context, the explainers to be executed, and their parameters are obtained from the `YAML` configuration file.

The optional `selected_explainers` argument can be used to restrict the execution to a subset of the configured explanation methods. In this example, it selects only the registered `"SHAP"` explainer, although this argument would not be necessary if SHAP were the only explainer defined in the configuration.

```
from xai_metrics.runner import run_explanation

config_path = "examples/specific_examples/config.yaml"
results = run_explanation(
    selected_explainers=["SHAP"],
    config=config_path,
    attribution_output_dir=None,
)
```

The returned structure contains the experimental metadata and the paths of any attribution files generated during the execution. When no output directory is provided, the attribution values remain available in the returned result without being exported through the argument shown in the example.

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"
```

```yaml
explainers:
  - name: SHAP
    params:
      mode: classification
      algorithm: auto
      max_background_samples: 100
      output_index: 1
      random_state: 42
```

The paths to `X_background` and `y_background` provide the reference data used to initialize the SHAP explainer. The paths to `X_batch` and `y_batch` identify the observations and targets for which the explanations are generated.

In this configuration, `output_index` is set explicitly to (1). Consequently, the wrapper always selects the SHAP values corresponding to output (1), even though `y_batch` is available in the context.

5.3 BreakDown

BreakDown generates a local explanation by decomposing the prediction of an individual observation into a sequence of feature contributions [7]. Starting from a reference prediction, the method introduces the feature values of the explained observation one at a time and measures how the expected model output changes at each step.

Let

$$\pi\left(\pi_1, \pi_2, \ldots, \pi_d\right)$$

denote an ordering of the (d) input features, and let

$$S_j\left(\pi_1, \pi_2, \ldots, \pi_j\right)$$

represent the subset containing the first (j) features in that ordering. The contribution assigned to feature (π_j) can be expressed as:

$$\Delta_{\pi_j}(x) v_x\left(S_{j-1}\right),$$

where (x) is the observation being explained and $(v_x(S))$ represents the expected model output when the features in (S) are fixed to their values in (x). A common theoretical definition of this value function is:

$$v_x(S) \mathbb{E}\left[f(X) \mid X_S = x_S\right],$$

where (f) is the predictive model, (X_S) denotes the variables contained in (S), and (x_S) contains their values in the explained observation.

The resulting local decomposition can be written as:

$$f(x) \mathbb{E}\left[f(X)\right] + \sum_{j=1}^{d} \Delta_{\pi_j}(x),$$

where ($\mathbb{E}[f(X)]$) is the reference prediction and each ($\Delta_{\pi_j}(x)$) represents the change produced when feature (π_j) is incorporated after the preceding features in the selected order.

A positive contribution indicates that the feature increases the selected model output relative to the value obtained in the preceding step. Conversely, a negative contribution indicates that the feature decreases that output. The magnitude of the contribution represents the size of the change attributed to the feature within the sequential decomposition.

BreakDown does not average the contribution of each feature across all possible feature orderings. Consequently, the attribution assigned to a feature may depend on the order in which the variables are introduced. This effect is particularly relevant when features interact or are statistically dependent. Different orders may distribute the same total prediction difference differently across the input features, although the contributions still form an additive decomposition of the selected prediction.

The implementation provided by `XAIMetrics` uses the `Explainer` and `predict_parts()` functionality from the `DALEX` package. The DALEX explainer is constructed using the model, the background data stored in the `ExplainerContext`, the optional background targets, and a prediction function adapted to the configured task.

For every observation, the wrapper calls `predict_parts()` with `type="break_down"`. The contributions returned by DALEX are then converted into a fixed-size attribution vector whose positions follow the column order of `context.X_background`.

The parameters supported by the explainer can be summarized as follows:

- `mode (str)`: type of predictive task explained by BreakDown. If its value is `'classification"`, the wrapper uses the model's `predict_proba()` method. For any other value, it uses `predict()`. (*Default:* `'classification"`)
- `output_index (int)`: output column selected from the array returned by `predict_proba()` in classification mode. For binary classifiers, index 1 commonly corresponds to the positive class. (*Default:* `1`)
- `label (str)`: label assigned to the DALEX model explainer. If no value is provided, the name of the model class is used. This label identifies the model within the DALEX object but does not modify the generated feature contributions. (*Default:* model class name)
- `verbose (bool)`: whether DALEX should print additional information while constructing the model explainer. (*Default:* `False`)
- `precalculate (bool)`: whether DALEX should precalculate model diagnostics when the internal `dalex.Explainer` object is created. Enabling this option may increase the initialization cost but allows DALEX to prepare information reused by its explanation methods. (*Default:* `True`)
- `order (Any)`: order in which the features are introduced into the sequential decomposition. Since BreakDown contributions depend on this order, changing the parameter may change the individual attribution values. When no order is specified, the default ordering strategy provided by DALEX is used. (*Default:* `None`)
- `n_samples (int or None)`: number of samples passed to the `N` argument of `predict_parts()`. This value controls the sampling used internally by DALEX when estimating the quantities required by the explanation. When it is not provided, the default DALEX behaviour is used. (*Default:* `None`)

- `keep_distributions (bool)`: whether DALEX should retain the distributions of the estimated contributions. These distributions may contain additional diagnostic information, although the `XAIMetrics` wrapper only returns the final attribution matrix. (*Default:* `False`)
- `random_state (int)`: random seed passed to `predict_parts()`. Fixing this value improves the reproducibility of computations involving random sampling. (*Default:* `42`)
- `abs (bool)`: whether to convert the generated contributions into absolute values before returning them. When enabled, the direction of each contribution is discarded and only its magnitude is preserved. (*Default:* `False`)

The explainer accepts input observations represented as a `pandas.DataFrame`, a PyTorch `Tensor`, a `NumPy` array, or another array-like object. Data frames are reordered according to the columns of `X_background`. PyTorch tensors are detached, transferred to the CPU, and converted into `NumPy` arrays. One-dimensional inputs are reshaped automatically into a batch containing one observation.

The prediction function used by BreakDown depends on the configured mode. In classification mode, the implementation searches for a `predict_proba()` method. In any other mode, it searches for `predict()`. The required method is first searched directly on the supplied model and then on its internal `model` attribute. If it is unavailable in both locations, the explainer raises an `AttributeError`.

In classification mode, the prediction array is converted into a `NumPy` array. If it has two dimensions, the column specified through `output_index` is selected:

$$\hat{f}(x)\,\mathrm{predict_proba}(x)_{\texttt{output\_index}}.$$

If the prediction function already returns a one-dimensional array, its values are used directly. In regression mode, the complete output of `predict()` is passed to DALEX.

The `targets` argument accepted by `explain()` is not used by this explainer. The model output to decompose is controlled by the prediction function and, in classification mode, by `output_index`. Therefore, every observation in the same execution is explained with respect to the same selected output column.

The internal `dalex.Explainer` object is created lazily the first time a particular model is explained. The implementation caches one DALEX explainer for each model object using its identity. Subsequent calls with the same model reuse the cached object, avoiding repeated initialization with the background data.

DALEX returns the sequential decomposition as a data frame containing feature names, contributions, and auxiliary rows such as the initial reference value and final prediction. The wrapper ignores rows whose names begin with an underscore and extracts the contribution associated with each input feature.

Some DALEX rows may represent a feature using a string that includes both its name and value, for example:

$$\texttt{feature\_name = feature\_value}.$$

When a returned name does not exactly match a column in the background data, the wrapper extracts the text before `" = "` and uses it as the feature name. The corresponding contribution is then placed in the position defined by `context.X_background`.

If the same feature appears in more than one result row, its contributions are accumulated. Features that do not appear in the DALEX result retain an attribution value of (0). This conversion produces a fixed-size vector independently of the row structure returned by `predict_parts()`.

The explainer computes a separate BreakDown explanation for each observation in the input batch. Therefore, the computational cost increases approximately linearly with the number of observations to explain.

The explainer returns a two-dimensional `NumPy` array with one row for each explained observation and one column for each input feature:

$$A \in \mathbb{R}^{n \times d},$$

where (n) is the number of observations in the explained batch and (d) is the number of features in `X_background`.

When `abs=False`, the returned values preserve the direction of the contributions. When `abs=True`, the absolute-value operation is applied after all individual explanations have been converted into the attribution matrix.

The explainer is implemented through the `BreakDownExplainer` class, located in the module `xai_metrics.explainers.breakdown`. It is registered under the name `"BreakDown"` and can be executed directly using this class (Section 5.3.1) or as part of the automated explanation pipeline provided by `run_explanation()` (Section 5.3.2).

Both usage modes rely on the same `YAML` configuration file. The example generates BreakDown explanations for an Isolation Forest model on the MetroPT3 dataset. The explainer operates in classification mode, decomposes the probability corresponding to output (1), and uses a fixed random seed of (42). Since no explicit feature order is configured, the default ordering behaviour provided by DALEX is used.

Examples of both usage modes are available in the library under: `examples/specific_examples/explainers/BreakDown_examples.py`

5.3.1 Direct explanation using the BreakDownExplainer class

In the direct usage mode, the explanation context is constructed from the `config.yaml` file using the `ConfigController` class. This context contains the model, background data, optional background labels, observations to explain, target values, and execution device required by the explanation process.

After constructing the context, the `BreakDownExplainer` class is instantiated directly using the desired parameters. Calling the `explain()` method with the model and input batch returns the complete attribution matrix. Although the target values are passed in the example for consistency with the common explainer interface, they are not used by BreakDown.

```
from xai_metrics.config import ConfigController
from xai_metrics.explainers.breakdown import BreakDownExplainer

config_path = "examples/specific_examples/config.yaml"
context, metadata = ConfigController(config=config_path).build_explainers_context()
explainer = BreakDownExplainer(
    context=context,
    params={
        "mode": "classification",
```

```
        "output_index": 1,
        "random_state": 42,
    },
)

attributions = explainer.explain(
    model=context.model,
    inputs=context.X_batch,
    targets=context.y_batch,
)
```

The resulting array contains one row for every observation in `context.X_batch` and one column for every feature in `context.X_background`. The contributions preserve their signs because the `abs` parameter is not enabled.

The mean absolute attribution calculated in the complete example summarizes the average magnitude of the generated contributions. However, it should not be interpreted as a direct measure of explanation quality, since it depends on the scale of the selected model output and on the distribution of the contributions across the features.

5.3.2 Automated explanation using run_explanation()

The automated usage mode uses the `run_explanation()` function. The explanation context, the explainers to be executed, and their parameters are obtained from the `YAML` configuration file.

The optional `selected_explainers` argument can be used to restrict the execution to a subset of the configured explanation methods. In this example, it selects only the registered `"BreakDown"` explainer, although this argument would not be necessary if BreakDown were the only explainer defined in the configuration.

```
from xai_metrics.runner import run_explanation

config_path = "examples/specific_examples/config.yaml"
results = run_explanation(
    selected_explainers=["BreakDown"],
    config=config_path,
    attribution_output_dir=None,
)
```

The returned structure contains the experimental metadata and the paths of any attribution files generated during the execution. When no output directory is provided, the attribution values remain available in the returned result without being exported through the argument shown in the example.

The corresponding experimental configuration is defined as follows:

```
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  xai_method_name: "LIME"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"
```

```yaml
explainers:
  - name: BreakDown
    params:
      mode: classification
      output_index: 1
      random_state: 42
```

The paths to `X_background` and `y_background` provide the reference data used to initialize the DALEX model explainer. The paths to `X_batch` and `y_batch` identify the observations and targets associated with the explanation batch. However, `y_batch` does not determine the explained output in this implementation because BreakDown uses the fixed `output_index` selected in the configuration.

In the configuration shown above, `xai_method_name` has been changed to `"BreakDown"` so that the experimental metadata are consistent with the explanation method being executed.

5.4 MAPLE

MAPLE, or Model Agnostic Supervised Local Explanations, combines local linear modelling with a supervised neighbourhood defined by a tree ensemble [**?**]. The method uses the tree ensemble in two complementary ways: to identify the training observations that are most relevant to the observation being explained and to select the features used by the local linear explanation.

First, the predictive model is evaluated on the background data. A forest ensemble is then trained to approximate these model responses. For a new observation x, the relevance of a background observation x_i is determined by how frequently both observations fall into the same terminal leaf across the trees:

$$w_i(x) = \frac{1}{K} \sum_{k=1}^{K} \mathbb{I}\left[T_k(x_i) = T_k(x)\right],$$

where K is the number of trees, $T_k(x)$ is the terminal leaf reached by x in tree k, and $\mathbb{I}[\cdot]$ is the indicator function. Observations that frequently share leaves with x receive larger weights and form a supervised local neighbourhood adapted to the behaviour of the model.

MAPLE then fits a weighted local linear model using the selected features:

$$\widehat{\boldsymbol{\beta}}_x = \underset{\boldsymbol{\beta}}{\operatorname{argmin}} \left[\sum_{i=1}^{n} w_i(x) \left(\boldsymbol{\beta}^{\mathsf{T}} \widetilde{x}_i - f(x_i) \right)^2 + \lambda \left\| \boldsymbol{\beta} \right\|_2^2 \right],$$

where $\widetilde{x}_i$ contains the retained features and an intercept term, $f(x_i)$ is the response of the original predictive model, and λ controls the regularization of the local linear model.

The feature coefficients of this local model form the explanation for x. Therefore, MAPLE differs from LIME mainly in the definition of locality. LIME creates an unsupervised neighbourhood using random perturbations and a distance kernel, whereas MAPLE learns a supervised neighbourhood from the model responses and the partition structure of a tree ensemble.

The default high-level XAIMetrics workflow can execute MAPLE with an empty parameter dictionary. The canonical MAPLE implementation is characterized by the following internal parameters:

- `n_estimators (int)`: number of trees used by the forest ensemble that approximates the model responses and defines the supervised neighbourhood. (*Canonical MAPLE default:* `200`)
- `max_features (float, int, or str)`: number or proportion of features considered when searching for a split in the forest ensemble. (*Canonical MAPLE default:* `0.5`)
- `min_samples_leaf (int)`: minimum number of observations required in a terminal tree leaf. Larger values produce broader and smoother neighbourhoods, whereas smaller values produce more specific partitions. (*Canonical MAPLE default:* `10`)
- `regularization (float)`: ridge regularization coefficient applied to the weighted local linear model. Larger values shrink the local coefficients and may improve numerical stability at the cost of reducing their magnitude. (*Canonical MAPLE default:* `0.001`)

These values describe the underlying MAPLE learner. They are implementation-level options and should only be added as `YAML` keys when the installed `MAPLEExplainer` wrapper explicitly exposes them. The examples below use the defaults selected by the wrapper and therefore specify an empty parameter mapping.

The background data serve two purposes: they are used to obtain the responses of the original model and to train the forest ensemble that defines the local neighbourhoods. The quality and coverage of the background dataset consequently have a direct influence on the explanations.

MAPLE returns one coefficient vector for each explained observation. The intercept of the local linear model is not included in the attribution matrix, since the matrix contains one value per input feature. Positive and negative coefficients represent the local direction of influence of each feature according to the supervised neighbourhood.

The method may require a higher initial computational cost than LIME because the supervised neighbourhood model must be trained. Once this model has been constructed, however, it can be reused to explain several observations. The explanations may still be affected by correlated features, insufficient background coverage, or a forest ensemble that does not approximate the original model accurately.

The method is implemented through the `MAPLEExplainer` class, located in the module `xai_metrics.explainers.maple`. It can be executed directly using this class (Section 5.4.1) or through `run_explanation()` (Section 5.4.2).

5.4.1 Direct explanation using the MAPLEExplainer class

```
from xai_metrics.config import ConfigController
from xai_metrics.explainers.maple import MAPLEExplainer

config_path = "examples/specific_examples/config.yaml"

context, metadata = ConfigController(
    config=config_path
).build_explainers_context()

explainer = MAPLEExplainer(
    context=context,
    params={}
)

attributions = explainer.explain()
```

5.4.2 Automated explanation using run_explanation()

```python
from xai_metrics.runner import run_explanation

config_path = "examples/specific_examples/config.yaml"

results = run_explanation(
    config=config_path,
    attribution_output_dir=None
)
```

The corresponding configuration is defined as follows:

```yaml
context:
  dataset_name: "MetroPT3"
  model_name: "IForest"
  device: "cpu"
  model_path: "examples/models/MetroPT3/IForest/MetroPT3_IForest_seed_0.pkl"
  X_background_path: "examples/datasets/MetroPT3/X_background.csv"
  y_background_path: "examples/datasets/MetroPT3/y_background.csv"
  X_batch_path: "examples/datasets/MetroPT3/X_batch.csv"
  y_batch_path: "examples/datasets/MetroPT3/y_batch.csv"

explainers:
  - name: MAPLE
    params: {}
```

5.5 Executing several explanation methods

The automated workflow can execute several explanation methods from the same configuration. This makes it possible to generate their attribution matrices under homogeneous conditions and subsequently evaluate them using the metrics implemented in the library.

The following configuration executes the four available methods for every discovered dataset and model:

```yaml
context:
  device: "cpu"
  datasets_dir: "examples/datasets"
  models_dir: "examples/models"
  attributions_dir: "examples/attributions"

explainers:
  - name: LIME
    params:
      mode: classification
      random_state: 42
      num_samples: 5000
      labels: [1]
      distance_metric: euclidean

  - name: SHAP
    params:
      max_background_samples: 100
      abs: false
```

```yaml
  - name: BreakDown
    params: {}

  - name: MAPLE
    params: {}
```

The experiment is executed as follows:

```python
from xai_metrics.runner import run_explanation

results = run_explanation(
    config="examples/config_example.yaml",
    attribution_output_dir="examples/attributions"
)
```

The returned object contains the explanation contexts, generated attribution matrices, parameters used by each explanation method, skipped executions, and paths of the exported files. Each attribution file can subsequently be loaded by `run_evaluation()` to compare the methods using the same set of evaluation metrics.

When several methods are compared, the following experimental decisions should remain consistent:

- the observations included in `X_batch`;
- the model output or target class being explained;
- the preprocessing and feature order;
- the background dataset and its sampling procedure;
- the use of signed or absolute attributions; and
- the random seeds used by stochastic explanation methods.

Maintaining these conditions prevents differences in the experimental setup from being incorrectly interpreted as differences between explanation methods.

CHAPTER

6

Reporting bugs

Feel free to open an issue at Github if anything is not working as expected https://github.com/kdis-lab/XAIMetrics/issues. Merge request are also encouraged, it will be carefully reviewed and merged if everything is all right.

Bibliography

[1] Emrullah ŞAHiN, Naciye Nur Arslan, and Durmuş Özdemir. Unlocking the black box: an in-depth review on interpretability, explainability, and reliability in deep learning, 1 2025.

[2] In-On Wiratsin and Chaiyong Ragkhitwetsagul. Effectiveness of explainable artificial intelligence (xai) techniques for improving human trust in machine learning models: A systematic literature review. *IEEE Access*, 2025.

[3] Logan Cummins, Alexander Sommers, Somayeh Bakhtiari Ramezani, Sudip Mittal, Joseph Jabour, Maria Seale, and Shahram Rahimi. Explainable predictive maintenance: A survey of current methods, challenges and opportunities. *IEEE Access*, 12:57574–57597, 2024.

[4] Scott M Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems 30*, pages 4765–4774. Curran Associates, Inc., 2017.

[5] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13-17, 2016*, pages 1135–1144, 2016.

[6] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International conference on machine learning*, pages 3319–3328. PMLR, 2017.

[7] Alicja Gosiewska and Przemyslaw Biecek. Do not trust additive explanations. *arXiv preprint arXiv:1903.11420*, 2019.

[8] Jianlong Zhou, Amir H. Gandomi, Fang Chen, and Andreas Holzinger. Evaluating the quality of machine learning explanations: A survey on methods and metrics, 3 2021.

[9] Pedro Lopes, Eduardo Silva, Cristiana Braga, Tiago Oliveira, and Luís Rosado. Xai systems evaluation: A review of human and computer-centred methods, 10 2022.

[10] Umang Bhatt, Adrian Weller, and José MF Moura. Evaluating and aggregating feature-based model explanations. *arXiv preprint arXiv:2005.00631*, 2020.

[11] David Alvarez Melis and Tommi Jaakkola. Towards robust interpretability with self-explaining neural networks. *Advances in neural information processing systems*, 31, 2018.

[12] Sanjoy Dasgupta, Nave Frost, and Michal Moshkovitz. Framework for evaluating faithfulness of local explanations. In *International Conference on Machine Learning*, pages 4794–4815. PMLR, 2022.

[13] An-phi Nguyen and María Rodríguez Martínez. On quantitative aspects of model interpretability. *arXiv preprint arXiv:2007.07584*, 2020.

[14] Marek Pawlicki, Aleksandra Pawlicka, Federica Uccello, Sebastian Szelest, Salvatore D'Antonio, Rafał Kozik, and Michał Choraś. Evaluating the necessity of the multiple metrics for assessing explainable ai: A critical examination. *Neurocomputing*, 602, 10 2024.

[15] Anna Hedström, Leander Weber, Daniel Krakowczyk, Dilyara Bareeva, Franz Motzkus, Wojciech Samek, Sebastian Lapuschkin, and Marina M-C Höhne. Quantus: An explainable ai toolkit for responsible evaluation of neural network explanations and beyond. *Journal of Machine Learning Research*, 24(34):1–11, 2023.

[16] Prasad Chalasani, Jiefeng Chen, Amrita Roy Chowdhury, Xi Wu, and Somesh Jha. Concise explanations of neural networks using adversarial training. In *International conference on machine learning*, pages 1383–1391. PMLR, 2020.

[17] Chih-Kuan Yeh, Cheng-Yu Hsieh, Arun Suggala, David I Inouye, and Pradeep K Ravikumar. On the (in) fidelity and sensitivity of explanations. *Advances in neural information processing systems*, 32, 2019.

[18] Chirag Agarwal, Nari Johnson, Martin Pawelczyk, Satyapriya Krishna, Eshika Saxena, Marinka Zitnik, and Himabindu Lakkaraju. Rethinking stability for attribution-based explanations. *arXiv preprint arXiv:2203.06877*, 2022.